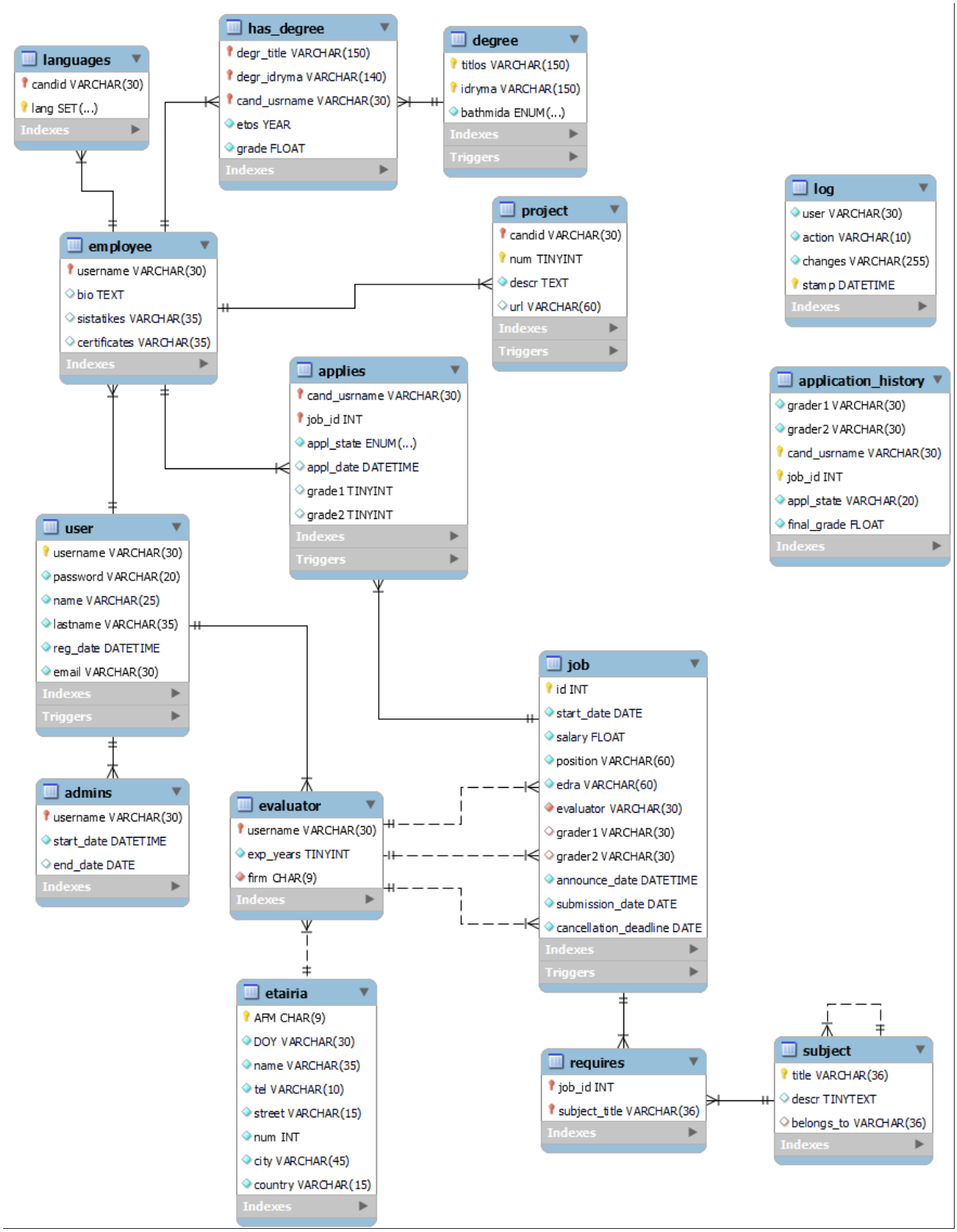
Κεφάλαιο 1

Σχεσιακό διάγραμμα:



Προπαρασκευαστική φάση:

Για την προπαρασκευαστική φάση εισήγαμε τον αριθμό εγγραφών που ζητούσε ο πίνακας της εκφώνησης. Αυτό που χρειάστηκε να προσέξουμε είναι ο περιορισμός του ότι για κάθε degree που υπάρχει στη βάση μας πρέπει αυτό το degree κάποιος να το έχει. Έτσι προσθέσαμε κατάλληλο πλήθος εγγραφών στον πίνακα has_degree και φτιάξαμε ένα trigger. Πριν γίνει κάποιο insert στον πίνακα has_degree το trigger ελέγχει αν ο συνδυασμός του νέου τίτλου και ιδρύματος υπάρχει στον πίνακα degree. Αν δεν υπάρχει τότε κάνει insert στον πίνακα degree το νέο τίτλο, το ίδρυμα και τη βαθμίδα BSc (η βαθμίδα ήταν αυθαίρετη επιλογή αφού δεν μπορούμε να την ξέρουμε από το has_degree). Επίσης αυτό γίνεται πριν το insert γιατί υπάρχουν ξένα κλειδιά προς το degree και αν το κάναμε μετά θα υπήρχε error.

Νέες απαιτήσεις:

- **Να δημιουργηθεί μηχανισμός για την εισαγωγή και την διαχείριση των αιτήσεων προαγωγής.**

Για αυτή την απαίτηση προσθέσαμε μια στήλη στον πίνακα applies με όνομα appl_state η οποία είναι ENUM με τιμές Active, Cancelled, Completed και DEFAULT τιμή την Active. Επίσης προσθέσαμε στον πίνακα job δύο παραγόμενες στήλες με ονόματα submission_date και cancellation_deadline οι οποίες είναι τύπου DATE και ορίζονται ως start_date – 15 και start_date – 10 αντίστοιχα. Για την αφαίρεση χρησιμοποιήσαμε τη συνάρτηση DATE_SUB. Για το περιορισμό που ο εργαζόμενος έχει ήδη τρεις ενεργές αιτήσεις φτιάξαμε trigger που θα παρουσιάσουμε παρακάτω.

- **Να δημιουργηθεί μηχανισμός για την αξιολόγηση και την εξαγωγή αποτελεσμάτων των αιτήσεων προαγωγής.**

Για την συγκεκριμένη απαίτηση θεωρήσαμε αναγκαία την προσθήκη των πεδίων:

- grade1, grade2, appl_date στον πίνακα applies
- grader1, grader2 στον πίνακα job

Θεωρούμε πως σε κάθε δουλειά που προκηρύσσεται, αναθέτονται 2 evaluators (grader1, grader2) που είναι υπεύθυνοι για την αξιολόγηση όλων των αιτήσεων για την συγκεκριμένη δουλειά. Κάθε αίτηση δέχεται και 2 βαθμολογίες (grade1, grade2) ο μέσος όρος των οποίων, καθορίζει τον πιο κατάλληλο employee για την δουλειά. Τα grade1 και grade2 είτε προέρχονται από τους grader1 και grader2, είτε υπολογίζονται αυτόματα από το σύστημα (procedure 3.1.3.1).

Τα πεδία grade1 και grade2 τα ορίσαμε ως TINYINT καθώς θα παίρνουν τιμές από 1 έως 20. Οι graders καθώς πρόκεινται για evaluators είναι VARCHAR(30) και το appl_date είναι τύπου datetime που έχει ως default τιμή το current_timestamp.

Για να εξασφαλίσουμε πως οι βαθμοί θα είναι από 1 έως 20 ορίσαμε 2 CHECK CONSTRAINTS τα εμφανίζουν error σε περίπτωση που πρόκειται να εισαχθεί τιμή εκτός αυτού του διαστήματος.

- **Να δημιουργηθεί ιστορικό αιτήσεων.**

Για αυτή την απαίτηση προσθέσαμε ένα καινούριο πίνακα στη βάση, με όνομα application_history. Οι στήλες του είναι ακριβώς όπως περιγράφονται από την εκφώνηση. Η στήλη appl_state που είναι η κατάσταση

της αίτησης έχει DEFAULT τιμή Completed. Εδώ κάναμε τη σχεδιαστική επιλογή να αφαιρέσουμε την τιμή Completed από το ENUM της στήλης appl_state του πίνακα applies αφού οποιαδήποτε αίτηση με κατάσταση Completed μπορεί να υπάρχει μόνο στο application_history. Επίσης ορίσαμε ως σύνθετο πρωτεύων κλειδί τις στήλες cand_usrname και job_id όπως και στον πίνακα applies. Προσθέσαμε ένα check constraint έτσι ώστε ο βαθμός να είναι ανάμεσα στο 1 και το 20. Ο πίνακας αυτός δεν σχετίζεται με κάποιον άλλο τρόπο με τους υπόλοιπους. Οι εγγραφές που εισήγαμε είναι 140.000.

- **DBA**

Για αυτή την απαίτηση προσθέσαμε δύο καινούριους πίνακες στη βάση, με ονόματα admins και log. Οι στήλες του πίνακα admins είναι ακριβώς όπως περιγράφονται από την εκφώνηση, με τη στήλη start_date να έχει DEFAULT τιμή CURRENT_TIMESTAMP. Το πρωτεύων κλειδί είναι το username και είναι ταυτόχρονα ξένο κλειδί στο username του πίνακα user. Τον πίνακα log τον παρουσιάζουμε παρακάτω.

Κεφάλαιο 2

3.1.3.1

Αρχικά, χρειάζεται να αναφέρουμε πως η procedure 3.1.3.1 που την ονομάσαμε grading() περιλαμβάνει και μια βοηθητική procedure την calcGrade() η οποία απλά εκτελεί τον τύπο που περιγράφεται στον πίνακα 2 της απαίτησης 3.1.2.2 της εκφώνησης. Επιλέξαμε να ορίσουμε και αυτή την συνάρτηση για λόγους οργάνωσης και για πιο ευανάγνωστο κώδικα.

Παράδειγμα εκτέλεσης της grading()

	cand_usrname varchar(30)	job_id int	* appl_state enum('Active	appl_date datetime	grade1 tinyint	grade2 tinyint
1	iMaxinq	9	Active	2023-12-31 13:59:38	9	(NULL)
2	Nikolis54	9	Active	2023-12-31 13:59:41	6	10
3	Agaphmemnonas21	8	Cancelled	2023-12-31 13:58:46	(NULL)	(NULL)
4	ChrisP	7	Active	2023-12-31 13:58:46	(NULL)	(NULL)
5	LokoLuke	5	Cancelled	2023-12-31 13:58:46	(NULL)	(NULL)
6	iMaxinq	4	Active	2023-12-31 13:58:46	(NULL)	(NULL)
7	iMaxinq	3	Active	2023-12-31 13:58:46	(NULL)	(NULL)
8	Marika365	1	Active	2023-12-31 13:58:46	(NULL)	(NULL)

Πίνακας applies

	id int	start_date date	salary float	position varchar(60)	edra varchar(60)	evaluator varchar(30)	grader1 varchar(30)	grader2 varchar(30)	announce_date datetime	submission_date date	cancellation_deadline date
1	1	2024-02-01	3500	Manager	Athens	Lenio666	(NULL)	(NULL)	2023-12-31 13:34:23	2024-01-17	2024-01-22
2	2	2024-02-03	3100	Mathematician	Patras	Riritos	(NULL)	(NULL)	2023-12-31 13:34:23	2024-01-19	2024-01-24
3	3	2024-09-15	1300	Software Engineer	Patras	GiovanniGeorgio	(NULL)	(NULL)	2023-12-31 13:34:23	2024-08-31	2024-09-05
4	4	2024-01-14	1600	Software Engineer	Thessaloniki	Lenio666	(NULL)	(NULL)	2023-12-31 13:34:23	2023-12-30	2024-01-04
5	5	2024-11-18	1100	Accountant	Patras	BessyLoL	(NULL)	(NULL)	2023-12-31 13:34:23	2024-11-03	2024-11-08
6	6	2024-04-23	1100	Software Engineer	Thessaloniki	Riritos	(NULL)	(NULL)	2023-12-31 13:34:23	2024-04-08	2024-04-13
7	7	2024-04-01	2760	Software Engineer	Athens	Achilleas12	(NULL)	(NULL)	2023-12-31 13:34:23	2024-03-17	2024-03-22
8	8	2024-12-01	2100	Civil Engineer	Patras	Patro4x4	(NULL)	(NULL)	2023-12-31 13:34:23	2024-11-16	2024-11-21
9	9	2024-05-02	1300	Junior programmer	Athens	Lenio666	Achilleas12	BessyLoL	2023-12-31 13:34:28	2024-04-17	2024-04-22

Πίνακας job

1)
```
CALL grading('Patro4x4','iMaxinq',9,@output);
```

Αρχικά καλούμε την procedure βάζοντας ως ορίσματα έναν evaluator (Patro4x4) που δεν αξιολογεί τον employee του ορίσματος (iMaxinq) για την συγκεκριμένη δουλειά (job id =9). (Οι graders της δουλειάς με job id=9 είναι οι Achilleas12 και BessyLoL).

Η procedure βγάζει ως αποτέλεσμα 0 :

	@output bigint
1	0

2)
```
CALL grading('Achilleas12','Nikolis54',9,@output1);
```

```
CALL grading('BessyLoL','Nikolis54',9,@output2);
```

Έπειτα καλούμε την συνάρτηση 2 φορές για έναν employee (Nikolis54) ο οποίος έχει ήδη καταχωρημένους και τους δύο βαθμούς του. Και στις δυο περιπτώσεις οι evaluators που δίνουμε ως όρισμα αξιολογούν τον συγκεκριμένο employee για την συγκεκριμένη δουλειά.

Βλέπουμε πως η procedure επιστρέφει τον βαθμό που έχει βάλει ο κάθε grader αντίστοιχα:

	@output1 bigint	@output2 bigint
1	6	10

3)
```
CALL grading('Achilleas12','iMaxinq',9,@output1);
```

```
CALL grading('BessyLoL','iMaxinq',9,@output2);
```

Τέλος καλούμε την συνάρτηση 2 φορές για έναν employee (iMaxinq) ο οποίος έχει μόνο έναν βαθμό από τους δύο συμπληρωμένο.

Στην πρώτη κλήση επιστρέφει τον βαθμό του grader1 (Achilleas12) που είναι συμπληρωμένος και στην δεύτερη κλήση όπου ο grader δεν έχει υποβάλλει βαθμό, τον υπολογίζει με βάση τον πίνακα 2 της εκφώνησης.

	@output1 bigint	@output2 bigint
1	9	6

Ο βαθμός που υπολογίστηκε είναι σωστός καθώς ο συγκεκριμένος employee ξέρει ξένη γλώσσα, έχει συμμετάσχει σε 2 projects και έχει 1 PhD (1+2+3=6). Αυτά φαίνονται από τους παρακάτω πίνακες project, languages, has_degree και degree αντίστοιχα.

	candid varchar(30)	num tinyint	* descr text	url varchar(60)
1	Agaphmemnonas21	1	Designed the blueprint for	www.github.com/Agaphmemnonas21/code
2	ChrisP	1	Made the company's websi	www.github.com/ChrisP/code/
3	iMaxinq	1	Created a database for a tra	www.github.com/iMaxinq/code/
4	iMaxinq	2	Mathematical predictions fo	www.github.com/iMaxinq/code/
5	LokoLuke	1	Statistical analysis of investi	www.github.com/LokoLuke/code/
6	Marika365	1	Took the necessary actions	www.github.com/Marika365/code/
7	Nikolis54	1	Did the calculations for a re	www.github.com/Nikolis54/code/

	candid varchar(30)	lang set('EN','FR','SP','GE
1	Agaphmemnonas21	CH,GR
2	iMaxinq	EN,FR,SP,GE,GR
3	LokoLuke	GR
4	Nikolis54	EN,SP,GE,GR

	degr_title varchar(150)	degr_idryma varchar(140)	cand_usrname varchar(30)	* etos year	* grade float
1	Business Administration	University of Athens	Marika365	2017	7.6
2	Civil Engineering	University of Thessaloniki	Agaphmemnonas21	2014	7.4
3	Computer Engineering and	University of Patras	iMaxinq	2021	8.3
4	Economics	University of Athens	LokoLuke	2013	10
5	Mathematics	University of Thessaloniki	Nikolis54	2016	7.4
6	Physics	University of Thessaloniki	Nikolis54	2017	8.9

	titlos varchar(150)	idryma varchar(150)	* bathmida enum('BSc'
1	Business Administration	University of Athens	MSc
2	Civil Engineering	University of Thessaloniki	MSc
3	Computer Engineering and	University of Patras	PhD
4	Economics	University of Athens	MSc
5	Mathematics	University of Thessaloniki	BSc
6	Physics	University of Thessaloniki	PhD

Κώδικας της grading():

```
DROP PROCEDURE IF EXISTS calcGrade;
DELIMITER $$
CREATE PROCEDURE calcGrade(IN eval_username_par VARCHAR(30), IN empl_username_par
VARCHAR(30), IN job_parameter  INT, OUT grade_parameter INT)
BEGIN
```

```
    DECLARE known_languages VARCHAR(30);
    DECLARE project_count TINYINT;
    DECLARE bsc_count TINYINT(4);
    DECLARE msc_count TINYINT(4);
    DECLARE phd_count TINYINT(4);

    SELECT MAX(num) INTO project_count
    FROM job INNER JOIN applies ON job.id=applies.job_id INNER JOIN employee ON
applies.cand_usrname=employee.username INNER JOIN
    project ON employee.username=project.candid
    WHERE job_parameter=applies.job_id AND (eval_username_par=job.grader1 OR
eval_username_par=job.grader2) AND empl_username_par=applies.cand_usrname;

    SELECT lang INTO known_languages
    FROM job INNER JOIN applies ON job.id=applies.job_id INNER JOIN employee ON
applies.cand_usrname=employee.username INNER JOIN
    languages ON employee.username=languages.candid
    WHERE job_parameter=applies.job_id AND (eval_username_par=job.grader1 OR
eval_username_par=job.grader2) AND empl_username_par=applies.cand_usrname;

    SELECT COUNT(bathmida) INTO bsc_count
    FROM job INNER JOIN applies ON job.id=applies.job_id INNER JOIN employee ON
applies.cand_usrname=employee.username
    INNER JOIN has_degree ON employee.username=has_degree.cand_usrname INNER JOIN degree ON
has_degree.degr_title=degree.titlos AND has_degree.degr_idryma=degree.idryma
    WHERE job_parameter=applies.job_id AND (eval_username_par=job.grader1 OR
eval_username_par=job.grader2) AND
    empl_username_par=applies.cand_usrname AND degree.bathmida='BSc'
    GROUP BY bathmida;

    SELECT COUNT(bathmida) INTO msc_count
    FROM job INNER JOIN applies ON job.id=applies.job_id INNER JOIN employee ON
applies.cand_usrname=employee.username
    INNER JOIN has_degree ON employee.username=has_degree.cand_usrname INNER JOIN degree ON
has_degree.degr_title=degree.titlos AND has_degree.degr_idryma=degree.idryma
    WHERE job_parameter=applies.job_id AND (eval_username_par=job.grader1 OR
eval_username_par=job.grader2) AND
    empl_username_par=applies.cand_usrname AND degree.bathmida='MSc'
    GROUP BY bathmida;

    SELECT COUNT(bathmida) INTO phd_count
    FROM job INNER JOIN applies ON job.id=applies.job_id INNER JOIN employee ON
applies.cand_usrname=employee.username
    INNER JOIN has_degree ON employee.username=has_degree.cand_usrname INNER JOIN degree ON
has_degree.degr_title=degree.titlos AND has_degree.degr_idryma=degree.idryma
    WHERE job_parameter=applies.job_id AND (eval_username_par=job.grader1 OR
eval_username_par=job.grader2) AND
    empl_username_par=applies.cand_usrname AND degree.bathmida='PhD'
    GROUP BY bathmida;
```

```
    IF (INSTR(known_languages,',')>0) THEN
        SET
grade_parameter=IFNULL(project_count,0)+IFNULL(bsc_count,0)+2*IFNULL(msc_count,0)+3*IFNULL(ph
d_count,0)+1;
    ELSE
        SET
grade_parameter=IFNULL(project_count,0)+IFNULL(bsc_count,0)+2*IFNULL(msc_count,0)+3*IFNULL(ph
d_count,0);
    END IF;
END $$
DELIMITER ;

--
##########################################################################################
################################################################## --

DROP PROCEDURE IF EXISTS grading;
DELIMITER $$
CREATE PROCEDURE grading(IN eval_username VARCHAR(30), IN empl_username VARCHAR(30), IN
job_param  INT, OUT grade_param INT)
BEGIN
    DECLARE vathmologitis1 VARCHAR(30);
    DECLARE vathmologitis2 VARCHAR(30);
    DECLARE vathmos1 TINYINT(4);
    DECLARE vathmos2 TINYINT(4);

    SELECT grader1,grader2,grade1,grade2 INTO vathmologitis1,vathmologitis2,vathmos1,vathmos2
    FROM job INNER JOIN applies ON job.id=applies.job_id
    WHERE  job_param=applies.job_id AND (eval_username=job.grader1 OR
eval_username=job.grader2) AND empl_username=applies.cand_usrname;

    IF (vathmologitis1=eval_username) THEN
        IF (vathmos1 IS NOT NULL) THEN
            SET grade_param=vathmos1;
        ELSE
            CALL calcGrade(eval_username,empl_username,job_param,grade_param);
        END IF;

    ELSEIF (vathmologitis2=eval_username) THEN
        IF (vathmos2 IS NOT NULL) THEN
            SET grade_param=vathmos2;
        ELSE
            CALL calcGrade(eval_username,empl_username,job_param,grade_param);
        END IF;

    ELSE
        SET grade_param=0;
    END IF;
END $$
```

3.1.3.2

Η συνάρτηση αυτή ονομάστηκε applyProcedure.
Η υλοποίηση είναι απλή και ευανάγνωστη:

```
DROP PROCEDURE IF EXISTS applyProcedure;
DELIMITER $
CREATE PROCEDURE applyProcedure(IN p_username VARCHAR(30), IN p_jobID INT, IN charX
ENUM('i','c','a'))
BEGIN
    DECLARE temp_grader1 VARCHAR(30);
    DECLARE temp_grader2 VARCHAR(30);
    DECLARE temp_firm VARCHAR(9);
    DECLARE temp_candUsrname VARCHAR(30);
    DECLARE temp_jobID INT;
    DECLARE temp_state VARCHAR(12);


    CASE charX
        WHEN 'i' THEN
            SELECT grader1,grader2 INTO temp_grader1,temp_grader2
            from employee INNER JOIN applies on employee.username = applies.cand_usrname
            INNER JOIN job ON applies.job_id = job.id
            WHERE p_username = employee.username AND p_jobID = job.id;

            IF temp_grader1 IS NULL THEN
                IF temp_grader2 IS NULL THEN
                    UPDATE job SET grader1 = 'GiovanniGeorgio' ,grader2 = 'Lenio666' WHERE
id=p_jobID;
                ELSE
                    SELECT firm into temp_firm
                    FROM evaluator
                    WHERE evaluator.username = temp_grader2;

                    SELECT username INTO temp_grader1
                    FROM evaluator
                    WHERE evaluator.firm = temp_firm AND evaluator.username != temp_grader2
                    LIMIT 1;

                    UPDATE job SET grader1 = temp_grader1 WHERE id=p_jobID;
                END IF;
```

```
            ELSE
                IF temp_grader2 IS NULL THEN
                    SELECT firm into temp_firm
                    FROM evaluator
                    WHERE evaluator.username = temp_grader1;

                    SELECT username INTO temp_grader2
                    FROM evaluator
                    WHERE evaluator.firm = temp_firm AND evaluator.username != temp_grader1
                    LIMIT 1;

                    UPDATE job SET grader2 = temp_grader2 WHERE id=p_jobID;
                END IF;
            END IF;

            INSERT INTO applies(cand_usrname,job_id,appl_state)
            VALUES (p_username,p_jobID,'Active');

        WHEN 'c' THEN
            SELECT cand_usrname,job_id INTO temp_candUsrname,temp_jobID
            FROM applies
            WHERE cand_usrname = p_username AND applies.job_id = p_jobID AND appl_state !=
'Cancelled';

            IF temp_jobID IS NULL THEN
                SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Application doesnt exist or has been cancelled';
            END IF;

            UPDATE applies SET appl_state = 'Cancelled' WHERE cand_usrname = temp_candUsrname
AND job_id = temp_jobID;


        WHEN 'a' THEN
            SELECT cand_usrname,job_id INTO temp_candUsrname,temp_jobID
            FROM applies
            WHERE cand_usrname = p_username AND applies.job_id = p_jobID AND appl_state =
'Cancelled';

            IF temp_candUsrname IS NOT NULL THEN
                UPDATE applies SET appl_state = 'Active' WHERE cand_usrname =
temp_candUsrname AND job_id = temp_jobID;
                SELECT 'Successfully updated application' AS message;
            ELSE
                SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'No cancelled application exists';

            END IF;
```

```
    END CASE;
END $
DELIMITER ;
```

Εάν οι 2 graders είναι NULL, τότε βάζουμε 2 τυχαίους απο την ίδια εταιρεία. Τυχαία επιλέξαμε τους GiovanniGeorgio και Lenio666.

Σύμφωνα με την βάση μας, μπορούμε να καλέσουμε την applyProcedure ως εξής: CALL applyProcedure(employee,jobID,i/c/a).
Για παράδειγμα από τον πίνακα employee παίρνω τον bigMAN για την job 3. Έχω:
CALL applyProcedure('bigMAN',3,'I');
Για αρχή στη βάση, υπάρχει μονο ένα apply ήδη από τον bigMAN και είναι ACTIVE για την θέση 5. Τρέχω λοιπόν την παραπάνω εντολη και βλέπω στον πίνακα applies μια νεα αίτηση ACTIVE απο τον bigMAN για την θέση 3. Αν τρέξω ακριβώς την ίδια εντολή ξανα, θα μου βγάλει error όπως και θα έπρεπε αφού πάω να εισάγω ίδια εγγραφή σε εναν πίνακα που έχει PrimaryKEY τον συνδυασμό (employee,jobID).
Αν τρέξω την εντολή:
CALL applyProcedure('bigMAN',3,'c');

Τότε θα παρατηρήσω στον πίνακα applies την αλλαγή από ACTIVE σε cancelled αιτηση. Και αν ξανατρέξω ακριβώς την ίδια εντολή, θα μου πετάξει error με μήνυμα που να με ενημερώνει ότι είτε δεν υπάρχει αιτηση είτε έχει ήδη γίνει cancelled.

Με την ίδια λογική τρέχω εντολές όπως:
CALL applyProcedure('bigMAN',3,'a'); η οποία ενεργοποιεί επιτυχώς την cancelled αίτηση

CALL applyProcedure('bigMAN',3,'a'); Πετάει error όπως το έχουμε γράψει να κανει, ενημερώνοντας ότι δεν υπάρχει ακυρωμένη αίτηση για επαναενεργοποίηση.

Τα παραπάνω παραδείγματα λειτουργούν το ίδιο για οποιοδήποτε άτομο από τον πίνακα employee επιλεχθεί.

3.1.3.3

Όπως και στην procedure 3.1.3.1. έτσι και εδώ χρησιμοποιήσαμε μια βοηθητική procedure, την fillGrades() η οποία όταν καλείται βρίσκει όλους τους applicants για την δουλειά που δίνεται ως όρισμα και συμπληρώνει τις βαθμολογίες τους αν αυτές δεν υπάρχουν. Η συγκεκριμένη procedure χρειάζεται καθώς για να βρούμε τον πιο κατάλληλο applicant πρέπει όλοι οι applicants να έχουν βαθμούς για να κάνουμε την σύγκριση. Αντί να βάλουμε απλά τον κώδικα στην κύρια συνάρτηση findEmployee() επιλέξαμε να φτιάξουμε αυτή την βοηθητική συνάρτηση για πιο ευανάγνωστο κώδικα.

<u>Παράδειγμα εκτέλεσης της findEmployee()</u>

	cand_usrname varchar(30)	job_id int	appl_state enum('Active	appl_date datetime	grade1 tinyint	grade2 tinyint
1	ChrisP	9	Active	2023-12-31 19:43:38	(NULL)	(NULL)
2	iMaxinq	9	Active	2023-12-31 19:43:27	9	(NULL)
3	LokoLuke	9	Active	2023-12-31 19:44:18	8	8
4	Marika365	9	Cancelled	2023-12-31 19:43:40	19	18
5	Nikolis54	9	Active	2023-12-31 19:43:29	6	10
6	Agaphmemnonas21	8	Cancelled	2023-12-31 19:42:51	(NULL)	(NULL)
7	ChrisP	7	Active	2023-12-31 19:42:51	(NULL)	(NULL)
8	LokoLuke	5	Cancelled	2023-12-31 19:42:51	(NULL)	(NULL)
9	iMaxinq	4	Active	2023-12-31 19:42:51	(NULL)	(NULL)
10	iMaxinq	3	Active	2023-12-31 19:42:51	(NULL)	(NULL)
11	Marika365	1	Active	2023-12-31 19:42:51	(NULL)	(NULL)

Πίνακας applies

Όπως φαίνεται στον πίνακα με τις προς επεξεργασία αιτήσεις, για την δουλειά με id=9 έχουν κάνει 5 άτομα αίτηση. Η μία από αυτούς (Marika365) έχει ακυρώσει την αίτησή της (Αυτό το αναφέρουμε καθώς θα πρέπει αυτός ο employee να τοποθετηθεί στο ιστορικό των αιτήσεων με βαθμό 0 και να μην ληφθεί υπόψιν στους υπολογισμούς του καταλληλότερου employee. Η Marika365 έχει πολύ μεγάλους βαθμούς και λογικά θα έπρεπε αυτή να επιλέγει αλλά θα δούμε ότι η procedure αγνοεί τις ακυρωμένες αιτήσεις).

```
CALL findEmployee(9);
```

Καλούμε την συνάρτηση findEmployee() για την δουλειά με id=9 η οποία επιστρέφει τον καταλληλότερο applicant για την δουλειά αυτή:

	cand_usrname varchar	grade newdecima
1	Nikolis54	8.0000

Παρόλο που ο Nikolis54 και ο LokoLuke έχουν τον ίδιο μέσο όρο, ο Nikolis54 έκανε πρώτος αίτηση όπως φαίνεται στο appl_date και άρα επιλέγεται αυτός.

Παράλληλα η συνάρτηση αφαιρεί τους applicants από τον πίνακα applies (πίνακας με τις προς επεξεργασία αιτήσεις):

	cand_usrname varchar(30)	job_id int	appl_state enum('Active	appl_date datetime	grade1 tinyint	grade2 tinyint
1	Agaphmemnonas21	8	Cancelled	2023-12-31 19:42:51	(NULL)	(NULL)
2	ChrisP	7	Active	2023-12-31 19:42:51	(NULL)	(NULL)
3	iMaxinq	3	Active	2023-12-31 19:42:51	(NULL)	(NULL)
4	iMaxinq	4	Active	2023-12-31 19:42:51	(NULL)	(NULL)
5	LokoLuke	5	Cancelled	2023-12-31 19:42:51	(NULL)	(NULL)
6	Marika365	1	Active	2023-12-31 19:42:51	(NULL)	(NULL)

Πίνακας applies

Και έπειτα προσθέτει τους applicants αυτούς στο ιστορικό αιτήσεων. Παρατηρούμε πως η employee με ακυρωμένη αίτηση έχει λάβει βαθμό 0 και οι υπόλοιποι έχουν τον μέσο όρο των grade1 και grade2.

grader1 varchar(30)	grader2 varchar(30)	cand_usrnam varchar(30)	job_id int	appl_state varchar(20)	final_grade float
Achilleas12	BessyLoL	ChrisP	9	Completed	1
Achilleas12	BessyLoL	iMaxinq	9	Completed	7.5
Achilleas12	BessyLoL	LokoLuke	9	Completed	8
Achilleas12	BessyLoL	Marika365	9	Completed	0
Achilleas12	BessyLoL	Nikolis54	9	Completed	8

Πίνακας application_history

Κώδικας της find_employee():

```
DROP PROCEDURE IF EXISTS fillGrades;
DELIMITER $$
CREATE PROCEDURE fillGrades(IN input_job INT)
BEGIN

    DECLARE finished INT DEFAULT 0;
    DECLARE applicant VARCHAR(30);
    DECLARE evaluator1 VARCHAR(30);
    DECLARE evaluator2 VARCHAR(30);
    DECLARE calculated_grade1 TINYINT(4);
    DECLARE calculated_grade2 TINYINT(4);

    DECLARE cursor_applicants CURSOR FOR
    SELECT cand_usrname,grader1,grader2
    FROM applies INNER JOIN job ON job.id=applies.job_id
```

```
    WHERE applies.job_id=input_job AND applies.appl_state = 'Active' AND (applies.grade1 IS
NULL OR applies.grade2 IS NULL);

    DECLARE CONTINUE HANDLER FOR NOT FOUND
    SET finished=1;

    OPEN cursor_applicants;
    REPEAT

        FETCH cursor_applicants INTO applicant,evaluator1,evaluator2;
        CALL grading(evaluator1,applicant,input_job,calculated_grade1);
        CALL grading(evaluator2,applicant,input_job,calculated_grade2);

        UPDATE applies
        SET grade1=calculated_grade1, grade2=calculated_grade2
        WHERE cand_usrname=applicant AND job_id=input_job;

    UNTIL finished=1
    END REPEAT;
    CLOSE cursor_applicants;
END $$
DELIMITER ;

--
##########################################################################################
################################################################## --

DROP PROCEDURE IF EXISTS findEmployee;
DELIMITER $$
CREATE PROCEDURE findEmployee(IN job_ID INT)
BEGIN

    CALL fillGrades(job_ID);

    SELECT cand_usrname,(grade1+grade2)/2 AS grade
    FROM applies
    WHERE applies.job_id = job_ID AND applies.appl_state = 'Active'
    ORDER BY grade DESC,appl_date ASC
    LIMIT 1;

    INSERT INTO application_history (cand_usrname,grader1,grader2,job_id,final_grade)
        SELECT
applies.cand_usrname,job.grader1,job.grader2,applies.job_id,(applies.grade1+applies.grade2)/2
        FROM applies INNER JOIN job ON job.id=applies.job_id
        WHERE applies.job_id=job_ID AND applies.appl_state = 'Active';

    INSERT INTO application_history (cand_usrname,grader1,grader2,job_id,final_grade)
        SELECT applies.cand_usrname,job.grader1,job.grader2,applies.job_id,0
        FROM applies INNER JOIN job ON job.id=applies.job_id
        WHERE applies.job_id=job_ID AND applies.appl_state = 'Cancelled';
```

```
    DELETE FROM applies
    WHERE  applies.job_id=job_ID;
END $$
DELIMITER ;
```

3.1.3.4

α)

```
DROP PROCEDURE IF EXISTS gradeRange;

DELIMITER $

CREATE PROCEDURE gradeRange(IN grade1 FLOAT, IN grade2 FLOAT)
BEGIN
    SELECT cand_usrname AS applicant, job_id AS job
    FROM application_history
    WHERE final_grade BETWEEN grade1 AND grade2;
END$
DELIMITER ;
```

<u>Παράδειγμα εκτέλεσης:</u>

Αρχικός πίνακας application_history:

application_history

Search results | Cost: 1ms | 1 | Total 10

	grader1 varchar(30)	grader2 varchar(30)	cand_usrnam varchar(30)	job_id int	appl_state varchar(20)	final_grade float
1	firsteval1	seceval1	Candidate1	1	Completed	2
2	firsteval10	seceval10	Candidate10	10	Completed	11
3	firsteval2	seceval2	Candidate2	2	Completed	3
4	firsteval3	seceval3	Candidate3	3	Completed	4
5	firsteval4	seceval4	Candidate4	4	Completed	5
6	firsteval5	seceval5	Candidate5	5	Completed	6
7	firsteval6	seceval6	Candidate6	6	Completed	7
8	firsteval7	seceval7	Candidate7	7	Completed	8
9	firsteval8	seceval8	Candidate8	8	Completed	9
10	firsteval9	seceval9	Candidate9	9	Completed	10

Αποτέλεσμα της gradeRange(2,5):

Result

Search results

	applicant varchar	job int
1	Candidate1	1
2	Candidate2	2
3	Candidate3	3
4	Candidate4	4

Αποτέλεσμα της gradeRange(10,15):

Result1

Search results

	applicant varchar	job int
1	Candidate10	10
2	Candidate9	9

β)

```
DROP PROCEDURE IF EXISTS evaluatedBy;

DELIMITER $

CREATE PROCEDURE evaluatedBy(IN eval_usrname VARCHAR(30))
BEGIN
    SELECT cand_usrname AS applicant, job_id AS job
    FROM application_history
    WHERE grader1 = eval_usrname OR grader2 = eval_usrname;
END$
DELIMITER ;
```

<u>Παράδειγμα εκτέλεσης:</u>

Με ίδιο αρχικό πίνακα έχουμε:

Αποτέλεσμα της evaluatedBy('seceval10'):

Result1

Search results

	applicant varchar	job int
1	Candidate10	10

Αποτέλεσμα της evaluatedBy('seceval8'):

Result1

Search results

	applicant varchar	job int
1	Candidate8	8

INDEXES

A)

```
ALTER TABLE application_history
ADD INDEX grade_idx (final_grade);
```

B)

```
ALTER TABLE application_history
ADD INDEX grader_idx (grader1,grader2);
```

Με τις παραπάνω εντολές ορίζουμε τα indexes στα πεδία final_grade και grader1,2 για τα ερωτήματα α) και β) αντίστοιχα. Τα συγκεκριμένα πεδία βρίσκονται στα WHERE conditions των procedures που ζητήθηκαν και βάση αυτών γίνεται η αναζήτηση του SELECT. Για αυτό τα ορίζουμε ως indexes, για να επιταχύνουμε την αναζήτηση.

Σχετικά με τους χρόνους αναζήτησης για 60 χιλιάδες εγγραφές, δεν παρατηρήσαμε κάποια διαφορά στον χρόνο. Για αυτό και αυξήσαμε τον αριθμό των εγγραφών στις 140 χιλιάδες όπου και πάλι δεν παρατηρήσαμε κάποια διαφορά.

Προσπαθήσαμε χωρίς επιτυχία να προσθέσουμε και άλλες εγγραφές καθώς και το mySQL Workbench και το vsCode δεν εκτελούσαν τα inserts και δεν ανταποκρίνονταν.

Κεφάλαιο 3

3.1.4.1

- **Triggers που θα ενημερώνουν το σχετικό πίνακα καταγραφής ενεργειών (log)**

Ο πίνακας log έχει τις στήλες id, user, action, changes και stamp με πρωτεύων κλειδί το id με AUTO_INCREMENT. Οι έξτρα στήλες id, action και changes ήταν σχεδιαστική μας επιλογή για καλύτερη κατανόηση του πίνακα. Η στήλη action είναι τύπου VARCHAR και έχει να κάνει με την ενέργεια που εκτελεί ο εκάστοτε user. Μπορεί να είναι INSERT, UPDATE ή DELETE. Η στήλη changes είναι τύπου VARCHAR και έχει να κάνει με τις αλλαγές που έκανε ο χρήστης με ένα action. Στα insert είναι η νέα εγγραφή, στα delete είναι η παλιά εγγραφή και στα update είναι η παλιά και η καινούρια εγγραφή.

```
#################     JOB INSERT       ##########################
DROP TRIGGER IF EXISTS log_job_in_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_job_in_trig  AFTER INSERT ON job
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT(NEW.id,', ',
                         NEW.start_date,', ',
```

```
                        NEW.salary,', ',
                        NEW.position,', ',
                        NEW.edra,', ',
                        NEW.evaluator);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'INSERT',changes,CURRENT_TIMESTAMP);
END$

DELIMITER ;

#################     JOB UPDATE       ##########################
DROP TRIGGER IF EXISTS log_job_up_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_job_up_trig  AFTER UPDATE ON job
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT('Old: ',
                        OLD.id,', ',
                        OLD.start_date,', ',
                        OLD.salary,', ',
                        OLD.position,', ',
                        OLD.edra,', ',
                        OLD.evaluator);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'UPDATE',changes,CURRENT_TIMESTAMP);

    SET changes = CONCAT('New: ',
                        NEW.id,', ',
                        NEW.start_date,', ',
                        NEW.salary,', ',
                        NEW.position,', ',
                        NEW.edra,', ',
                        NEW.evaluator);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'UPDATE',changes,CURRENT_TIMESTAMP);
END$

DELIMITER ;

#################     JOB DELETE       ##########################
DROP TRIGGER IF EXISTS log_job_del_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_job_del_trig  AFTER DELETE ON job
FOR EACH ROW
```

```
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT(OLD.id,', ',
                         OLD.start_date,', ',
                         OLD.salary,', ',
                         OLD.position,', ',
                         OLD.edra,', ',
                         OLD.evaluator);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'DELETE',changes,CURRENT_TIMESTAMP);
END$

DELIMITER ;
#################     USER INSERT       ##########################
DROP TRIGGER IF EXISTS log_user_in_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_user_in_trig  AFTER INSERT ON user
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT(NEW.username,', ',
                         NEW.password,', ',
                         NEW.name,', ',
                         NEW.lastname,', ',
                         NEW.email);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'INSERT',changes,CURRENT_TIMESTAMP);
END$

DELIMITER ;

#################     USER UPDATE       ##########################
DROP TRIGGER IF EXISTS log_user_up_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_user_up_trig  AFTER UPDATE ON user
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT('Old: ',
                         OLD.username,', ',
                         OLD.password,', ',
                         OLD.name,', ',
                         OLD.lastname,', ',
                         OLD.email);

    INSERT INTO log(user,action,changes,stamp)
```

```
    VALUES(CURRENT_USER(),'UPDATE',changes,CURRENT_TIMESTAMP);

    SET changes = CONCAT('New: ',
                         NEW.username,', ',
                         NEW.password,', ',
                         NEW.name,', ',
                         NEW.lastname,', ',
                         NEW.email);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'UPDATE',changes,CURRENT_TIMESTAMP);
END$

DELIMITER ;

#################     USER DELETE       ##########################
DROP TRIGGER IF EXISTS log_user_del_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_user_del_trig  AFTER DELETE ON user
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT(OLD.username,', ',
                         OLD.password,', ',
                         OLD.name,', ',
                         OLD.lastname,', ',
                         OLD.email);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'DELETE',changes,CURRENT_TIMESTAMP);
END$

DELIMITER ;

#################     DEGREE INSERT       ##########################
DROP TRIGGER IF EXISTS log_degree_in_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_degree_in_trig  AFTER INSERT ON degree
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT(NEW.titlos,', ',
                         NEW.idryma,', ',
                         NEW.bathmida);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'INSERT',changes,CURRENT_TIMESTAMP);
END$
```

```
DELIMITER ;

#################     DEGREE UPDATE       ##########################
DROP TRIGGER IF EXISTS log_degree_up_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_degree_up_trig  AFTER UPDATE ON degree
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT('Old: ',
                         OLD.titlos,', ',
                         OLD.idryma,', ',
                         OLD.bathmida);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'UPDATE',changes,CURRENT_TIMESTAMP);

    SET changes = CONCAT('New: ',
                         NEW.titlos,', ',
                         NEW.idryma,', ',
                         NEW.bathmida);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'UPDATE',changes,CURRENT_TIMESTAMP);
END$

DELIMITER ;

#################     DEGREE DELETE       ##########################
DROP TRIGGER IF EXISTS log_degree_del_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_degree_del_trig  AFTER DELETE ON degree
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT(OLD.titlos,', ',
                         OLD.idryma,', ',
                         OLD.bathmida);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'DELETE',changes,CURRENT_TIMESTAMP);
END$

DELIMITER ;
```

Παραδείγματα για τα INSERT:

User:

```
INSERT INTO user(username,password,name,lastname,email)
VALUES('trigTestUsr','1234','test','user','testuser@users.db');
```

Log:

log

Search results | Cost: 2ms | 1 | Total 27

	id int	user varchar(30)	action varchar(10)	changes varchar(255)	stamp datetime
1	27	root@localhost	INSERT	trigTestUsr, 1234, test, user,	2023-12-31 13:55:21

Degree:

```
INSERT INTO degree(titlos,idryma,bathmida)
VALUES('trigTestTitlos','trigTestIdryma','MSc');
```

Log:

log

Search results | Cost: 2ms | 1 | Total 28

	id int	user varchar(30)	action varchar(10)	changes varchar(255)	stamp datetime
1	28	root@localhost	INSERT	trigTestTitlos, trigTestIdrym:	2023-12-31 13:58:49
2	27	root@localhost	INSERT	trigTestUsr, 1234, test, user,	2023-12-31 13:55:21

Job:

```
INSERT INTO job(start_date,salary,position,edra,evaluator)
VALUES('2025-05-05',500.0,'trigTestPos','Athens','kellyC');
```

Log:

log

Search results | Cost: 1ms | 1 | Total 29

	id int	user varchar(30)	action varchar(10)	changes varchar(255)	stamp datetime
1	29	root@localhost	INSERT	9, 2025-05-05, 500, trigTest	2023-12-31 13:59:41
2	28	root@localhost	INSERT	trigTestTitlos, trigTestIdrym:	2023-12-31 13:58:49
3	27	root@localhost	INSERT	trigTestUsr, 1234, test, user,	2023-12-31 13:55:21

Παραδείγματα για τα UPDATE:

User:

```
UPDATE user
SET username = 'updatedUrsname'
WHERE username = 'trigTestUsr';
```

Log:

log

Search results | Cost: 1ms | 1 | Total 31

	id int	user varchar(30)	actior varchai	changes varchar(255)	stamp datetime
1	32	root@localhost	UPDATE	Old: trigTestUsr, 1234, test, user, t	2023-12-31 14:08:52
2	33	root@localhost	UPDATE	New: updatedUrsname, 1234, test	2023-12-31 14:08:52

Degree:

```
UPDATE degree
SET titlos = 'updatedTitlos'
WHERE titlos = 'trigTestTitlos' AND idryma = 'trigTestIdryma';
```

Log:

log | degree

Search results | Cost: 3ms | 1 | Total 33

	id int	user varchar(30)	actior varchai	changes varchar(255)	stamp datetime
1	35	root@localhost	UPDATE	New: updatedTitlos, trigTestIdrym	2023-12-31 14:11:32
2	34	root@localhost	UPDATE	Old: trigTestTitlos, trigTestIdryma,	2023-12-31 14:11:32

Job:

```
UPDATE job
SET position = 'updatedPos'
WHERE id = 9;
```

Log:

log | job

Search results | Cost: 2ms | 1 | Total 35

	id int	user varchar(30)	actior varchai	changes varchar(255)	stamp datetime
1	37	root@localhost	UPDATE	New: 9, 2025-05-05, 500, updatedPos, A	2023-12-31 14:13:45
2	36	root@localhost	UPDATE	Old: 9, 2025-05-05, 500, trigTestPos, Ath	2023-12-31 14:13:45

Παραδείγματα για τα DELETE:

User:

```
DELETE FROM user
WHERE username = 'updatedUrsname';
```

Log:

log | user

Search results | Cost: 1ms | 1 | Total 36

id int	user varchar(30)	actior varchai	changes varchar(255)	stamp datetime
38	root@localhost	DELETE	updatedUrsname, 1234, test, user, testu:	2023-12-31 14:17:37

Degree:

```
DELETE FROM degree
WHERE titlos = 'updatedTitlos' AND idryma = 'trigTestIdryma';
```

Log:

log

Search results | Cost: 2ms | 1 | Total 37

id int	user varchar(30)	actior varchai	changes varchar(255)	stamp datetime
39	root@localhost	DELETE	updatedTitlos, trigTestIdryma, MSc	2023-12-31 14:18:18

Job:

```
DELETE FROM job
WHERE id = 9;
```

Log:

log

Search results | Cost: 2ms | 1 | Total 38

id int	user varchar(30)	actior varchai	changes varchar(255)	stamp datetime
40	root@localhost	DELETE	9, 2025-05-05, 500, updatedPos, Athens	2023-12-31 14:19:00

3.1.4.2

- **Trigger ο οποίος θα αποτρέπει την εισαγωγή αίτησης**

Το trigger αυτό το εφαρμόζουμε πριν τα insert στο πίνακα applies και ελέγχουμε δύο συνθήκες. Η πρώτη είναι αν η ημερομηνία εισαγωγής απέχει λιγότερο από 15 μέρες από την ημερομηνία έναρξης της θέσης εργασίας. Για να το κάνουμε αυτό κάνουμε SELECT το submission_date από τον πίνακα job όπου job.id είναι το job_id της νέας αίτησης. Αν ο περιορισμός ισχύει τότε κάνουμε SIGNAL με μήνυμα ότι η ημερομηνία αίτησης είναι πολύ κοντά στην ημερομηνία έναρξης. Αν δεν ισχύει τότε κάνουμε SELECT για να βρούμε πόσες ενεργές αιτήσεις έχει ο εργαζόμενος που υποβάλει την αίτηση. Αν αυτές είναι τρείς τότε κάνουμε SIGNAL με ανάλογο μήνυμα.

```
DROP TRIGGER IF EXISTS applies_in_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS applies_in_trig  BEFORE INSERT ON applies
FOR EACH ROW
BEGIN
    DECLARE submission_date DATE;
    DECLARE active_applications TINYINT(1);

    ########    sub_date    ########
    SELECT job.submission_date INTO submission_date
    FROM job
    WHERE job.id = NEW.job_id;

    IF CURRENT_DATE > submission_date THEN # CURRENT_DATE is the date an applicant attempts
to apply
        SIGNAL SQLSTATE VALUE '45000'
        SET MESSAGE_TEXT = 'Application date too close to job start date.';
    END IF;

    ########    cand active applications    ########
    SELECT count(appl_state) INTO active_applications
    FROM applies
    WHERE appl_state = 'Active' AND cand_usrname = NEW.cand_usrname;

    IF active_applications = 3 THEN
        SIGNAL SQLSTATE VALUE '45000'
        SET MESSAGE_TEXT = 'Applicant already has three active applications.';
    END IF;
END$

DELIMITER ;
```

Αρχικά προσθέτουμε μια εγγραφή στον πίνακα job που να παραβιάζει το trigger αν κάνουμε apply.

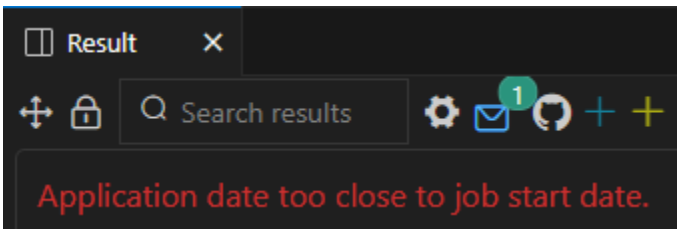
```
INSERT INTO job(start_date,salary,position,edra,evaluator)
VALUES('2024-01-05',500.0,'Clerk','Athens','kellyC');
```

	id int	start_date date	salary float	position varchar(60)	edra varchar(60)	evaluator varchar(30)	grader1 varchar(30)	grader2 varchar(30)	announce_date datetime	submission_date date	cancellation_deadline date
1	11	2024-01-05	500	Clerk	Athens	kellyC	(NULL)	(NULL)	2023-12-31 14:23:16	2023-12-21	2023-12-26

Στη συνέχεια θα κάνουμε INSERT στον πίνακα applies για το job που μόλις βάλαμε.

```
INSERT INTO applies(cand_usrname,job_id)
VALUES('markB',11);
```

Η εγγραφή δεν μπήκε στον πίνακα και το αποτέλεσμα είναι αυτό:



Για την δεύτερη συνθήκη θα κάνουμε ένα INSERT στον πίνακα applies έτσι ώστε ένας employee να έχει τρείς Active αιτήσεις.

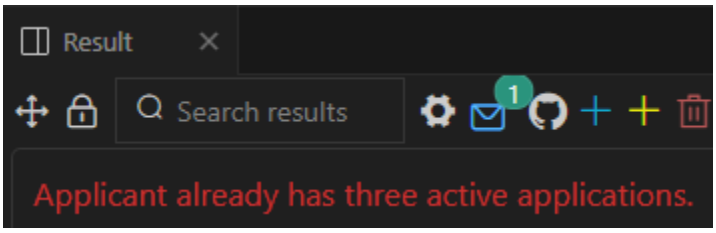
```
INSERT INTO applies(cand_usrname,job_id)
VALUES('SpirosP',7);
```

SpirosP	1	Active
SpirosP	3	Cancelled
SpirosP	4	Active
SpirosP	7	Active

Τώρα κάνουμε INSERT με το ίδιο username και διαφορετικό job id.

```
INSERT INTO applies(cand_usrname,job_id)
VALUES('SpirosP',9);
```

Η εγγραφή δεν μπήκε στον πίνακα και το αποτέλεσμα είναι αυτό:



3.1.4.3

- **Trigger ο οποίος θα αποτρέπει την ακύρωση μιας αίτησης και θα αποτρέπει την ενεργοποίηση ακυρωμένης αίτησης.**

Το trigger αυτό το εφαρμόζουμε πριν τα update στο πίνακα applies και ελέγχουμε τέσσερεις συνθήκες. Η πρώτη είναι αν το update που πάμε να κάνουμε είναι να θέσουμε το appl_state ίσο με Canceled. Αν αυτή ισχύει ελέγχουμε αν η ημερομηνία ενημέρωσης απέχει λιγότερο από 10 μέρες από την ημερομηνία έναρξης της θέσης εργασίας. Αν ναι τότε κάνουμε SIGNAL με μήνυμα ότι η ημερομηνία ακύρωσης είναι πολύ κοντά στην ημερομηνία έναρξης. Η τρίτη συνθήκη ελέγχεται μόνο αν το update που πάμε να κάνουμε είναι να θέσουμε το appl_state ίσο με Active σε αίτηση με προηγούμενη κατάσταση ίση με Canceled. Αν ισχύει τότε ελέγχουμε αν ο εργαζόμενος έχει ήδη τρείς ενεργές αιτήσεις όπως και στο προηγούμενο trigger και αν ναι τότε κάνουμε SIGNAL με ανάλογο μήνυμα.

```
DROP TRIGGER IF EXISTS applies_cancel_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS applies_cancel_trig  BEFORE UPDATE ON applies
FOR EACH ROW
BEGIN

    DECLARE cancelation_date DATE;
    DECLARE active_applications TINYINT(1);

    IF NEW.appl_state = 'Cancelled' THEN

        ########    cancel_date    ########
        SELECT job.cancelation_date INTO cancelation_date
        FROM job
        WHERE job.id = NEW.job_id;

        IF CURRENT_DATE > cancelation_date THEN # CURRENT_DATE is the date an applicant
attempts to cancel
            SIGNAL SQLSTATE VALUE '45000'
            SET MESSAGE_TEXT = 'Cancelation date too close to job start date.';
        END IF;
    END IF;

    IF NEW.appl_state = 'Active' AND OLD.appl_state = 'Cancelled' THEN

        ########    cand active applications    ########
        SELECT count(appl_state) INTO active_applications
        FROM applies
        WHERE appl_state = 'Active' AND cand_usrname = NEW.cand_usrname;

        IF active_applications = 3 THEN
            SIGNAL SQLSTATE VALUE '45000'
            SET MESSAGE_TEXT = 'Applicant already has three active applications.';
```

```
        END IF;
    END IF;
END$

DELIMITER ;
```

Στον πίνακα job θα κάνουμε UPDATE το start_date της εγγραφής που προσθέσαμε προηγουμένως για να βάλουμε μια Active αίτηση στον πίνακα applies για αυτή.

```
UPDATE job
SET start_date = '2024-02-05'
WHERE id = 11;
```

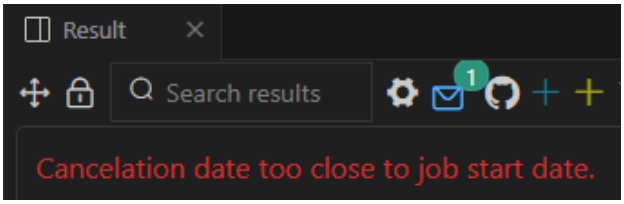
```
INSERT INTO applies(cand_usrname,job_id)
VALUES('markB',11);
```

Στη συνέχεια θα κάνουμε UPDATE τις εγγραφές στους πίνακες job και applies.

```
UPDATE job
SET start_date = '2024-01-05'
WHERE id = 11;
```

```
UPDATE applies
SET appl_state = 'Cancelled'
WHERE cand_usrname = 'MarkB' AND job_id = 11;
```

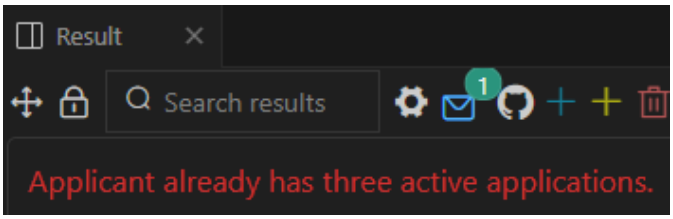
Η εγγραφή δεν άλλαξε στον πίνακα και το αποτέλεσμα είναι αυτό:



Για τη δεύτερη συνθήκη έχουμε ήδη στον πίνακα applies εγγραφές για να παραβιάζεται το trigger οπότε δεν χρειάζεται να κάνουμε καινούριες.

```
UPDATE applies
SET appl_state = 'Active'
WHERE cand_usrname = 'SpirosP' AND job_id = 3;
```

Η εγγραφή δεν άλλαξε στον πίνακα και το αποτέλεσμα είναι αυτό:



Κεφάλαιο 4

Για την υλοποίηση του GUI που ζητά η εκφώνηση χρησιμοποιήσαμε το IntelliJ IDEA. Το project αποτελείται από τις εξής κλάσεις:

- Main
- login
- userHandler
- DBHandler
- Update
- Insert

Main

Η κλάση αυτή αποτελείται από τρείς μεταβλητές που είναι απαραίτητες για τη σύνδεση με τη βάση. Αυτές είναι οι: JDBC_URL , DB_USER, DB_PASSWORD. Έχει επίσης την main method που καλεί μια ένα νέο αντικείμενο της κλάσης login.

login

Η κλάση αυτή υιοθετεί την κλάση JFrame και έχει ένα constructor, τρείς ActionListeners και τη μέθοδο private char credentialsCheck(String username, String password). Ο constructor ορίζει το βασικό frame και τα χαρακτηριστικά του με τη βοήθεια ενός Form που φτιάξαμε με το Swing UI Designer Tool του IntelliJ.

Υπάρχει ένα CLEAR JButton, ένα LOGIN JButton, ένα username JTextField και ένα password JPasswordField. Το ActionListener του CLEAR θέτει ένα κενό String στο στα δύο Fields. Το password έχει ένα Listener που εάν πατηθεί το Enter καλείται ο Listener του Login button. Ο listener του Login διαβάζει ότι υπάρχει στα δύο Fields και καλεί την credentialsCheck με αυτά. Η μέθοδος αυτή συνδέεται με τη βάση μας και εκτελεί ένα SELECT για να δει αν τα στοιχεία που δόθηκαν ανήκουν σε χρήστη που είναι admin. Αν όχι τότε επιστρέφει τον χαρακτήρα 'i' εννοώντας ότι τα στοιχεία είναι invalid. Αν ναι τότε επιστρέφει τον χαρακτήρα 'a' εννοώντας ότι τα στοιχεία είναι σωστά και ανήκουν σε admin. Ο Listener έχει ένα switch και ανάλογα με τον χαρακτήρα που επέστρεψε η credentialsCheck είτε πετάει JOptionPane που λέει ότι το username ή το password είναι λανθασμένα, είτε JOptionPane που λέει ότι η σύνδεση ήταν επιτυχής καλώντας αμέσως μετά ένα αντικείμενο DBHandler αφού κάνει dispose το login.

DBHandler

Η κλάση αυτή υιοθετεί την κλάση JFrame και έχει ένα constructor, τέσσερεις ActionListeners και τις εξής μεθόδους:

- public static boolean confirmation(boolean important)
- protected String whereConditions(List<String> primaryKeyColumns)
- private void addCB(JPanel txtFields, JTextField textField, String typeOfCB)
- protected void addTFnLabels(String tableName, JPanel txtFields, JTable selectedTableData, char action)
- private static JComboBox makeComboBoxes(String column)
- protected boolean allTextFieldsEmpty(char action)
- protected void clearTextFields()
- protected String[] makeRow()
- protected static List<String> getPrimaryKeyColumns(DatabaseMetaData metaData, String tableName)
- public static void displayTableData(JTable jTableName, String tableName, char user, String username)
- private static void fillComboBox(JComboBox tableCB)

Η κλάση έχει επίσης αυτά τα ArrayLists που θα χρησιμεύσουν αργότερα:

- public List<JTextField> textFieldList
- public List<String> labelList
- public List<JComboBox> comboBoxList

Ο constructor ορίζει το βασικό Frame και τα χαρακτηριστικά του με τη βοήθεια ενός Form. Επίσης καλεί την μέθοδο fillComboBox με όρισμα ένα JComboBox που υπάρχει στο Frame. Η μέθοδος αυτή συνδέεται με τη βάση και εκτελεί το query “SHOW TABLES” που μας επιστρέφει τα ονόματα των πινάκων της βάσης τα οποία στην συνέχεια προσθέτουμε στο ComboBox που περάσαμε ως όρισμα.

Ο πρώτος Listener είναι για το ComboBox που μόλις φτιάξαμε και καλείται όταν κάνουμε κάποια επιλογή. Με αυτή την επιλογή καλούμε τη μέθοδο displayTableData δίνοντας ως επιπλέον ορίσματα το JTable στο οποίο θα γίνει το display, το είδος user που είναι συνδεδεμένος και το όνομά του (τα τελευταία δύο είναι μέρος του μπόνους που θα παρουσιάσουμε στο κεφάλαιο 5).Η μέθοδος αυτή ανάλογα με τα ορίσματα που δώσαμε καλεί ένα SELECT και τυπώνει το αποτέλεσμά του στο JTable που θέλουμε.

Ο επόμενος Listener είναι για ένα κουμπί το οποίο χρησιμοποιείται για διαγραφή μιας εγγραφής από τη βάση. Αρχικά ελέγχει αν ο χρήστης έχει επιλέξει ακριβώς μια γραμμή και αν όχι τότε πετάει OptionPane που λέει είτε ότι ο πίνακας είναι κενός, είτε ότι πρέπει να επιλέξει μια ακριβώς γραμμή. Αν ναι τότε καλεί την μέθοδο confirmation με όρισμα false. Η μέθοδος αυτή πετάει ένα confirmation OptionPane με μήνυμα ανάλογο του ορίσματος important. Αν αυτό είναι true τότε το μήνυμα είναι μία έντονη προειδοποίηση, αλλιώς είναι μια απλή ερώτηση. Η μέθοδος επιστρέφει true μόνο αν ο χρήστης πατήσει το κουμπί YES ενώ σε κάθε άλλη περίπτωση επιστρέφει false. Αν επιστραφεί false τότε ο Listener σταματά. Διαφορετικά συνδέεται με τη βάση και ετοιμάζει ένα DELETE query με τη βοήθεια των μεθόδων getPrimaryKeyColumns και whereConditions.

Η μέθοδος getPrimaryKeyColumns παίρνει ως ορίσματα τα metadata της σύνδεσης με τη βάση, και το όνομα πίνακα για τον οποίο θέλουμε να βρούμε τα πρωτεύοντα κλειδιά. Δημιουργεί και επιστρέφει ένα List από Strings που γεμίζει με τα ονόματα των κλειδιών που επέστρεψε η μέθοδος getPrimaryKeys από τα matadata.

Η μέθοδος whereConditions παίρνει ως όρισμα μία λίστα από Strings (αυτή με τα πρωτεύοντα κλειδιά) και με ένα StringBuilder κατασκευάζει τα conditions του WHERE στο query που θα εκτελέσουμε και τα επιστρέφει ως ένα String.

Αφού ετοιμάσουμε το String του query θα φτιάξουμε ένα PreparedStatement και θα περάσουμε τις τιμές των πρωτευόντων κλειδιών της εγγραφής που θέλουμε να διαγράψουμε. Θα το εκτελέσουμε και στη συνέχεια θα αφαιρέσουμε την γραμμή από το JTable.

Οι επόμενοι Listeners έχουν ίδια δομή και είναι για ένα κουμπί insert και ένα update. Αρχικά κάνουν dispose το DBHandler και μετά καλούν την αντίστοιχη κλάση Insert ή Update περνώντας τους ως ορίσματα τον πίνακα που έχει επιλεγεί από το ComboBox.

Insert

Η κλάση αυτή υιοθετεί την κλάση DBHandler και έχει ένα constructor, δύο ActionListeners και την μέθοδο private String generateInsertQuery(String tableName, String[] rowData). Ο constructor ορίζει το βασικό Frame και τα χαρακτηριστικά του με τη βοήθεια ενός Form. Επίσης καλεί τις μεθόδους displayTableData με τα ορίσματα που έχουμε περιγράψει όπου το όνομα πίνακα είναι το όρισμα του constructor και την addTFnLabels.

Η μέθοδος addTFnLabels είναι η βασική μέθοδος που χρησιμοποιούμε και είναι υπεύθυνη για την δυναμική τοποθέτηση TextFields, Labels και ComboBoxes σε ένα JPanel από ένα JTable. Και τα δύο τα δίνουμε ως ορίσματα. Αρχικά εξάγουμε το tableModel από το JTable και ορίζουμε το Layout του Panel που θα γίνουν οι τοποθετήσεις. Ένα όρισμα της μεθόδου είναι ένα char action με βάση το οποίο προσθέτουμε τα πεδία που θέλουμε. Αν το action είναι 'i' δηλαδή insert δεν χρειάζεται να αποτυπώσουμε όλα τα πεδία κάθε πίνακα (π.χ. το id στον πίνακα job) οπότε ένα switch είναι υπεύθυνο για αυτό. Για τις στήλες που μας ενδιαφέρουν βάζουμε στο Panel ένα Label με το όνομά τους και ένα TextField. Σε στήλες που είναι ξένα κλειδιά όμως αντί για TextField βάζουμε ComboBox με το κατάλληλο περιεχόμενο. Αυτό το χειρίζεται ένα hardcoded switch που καλεί τη μέθοδο addCB με ορίσματα το Panel, ένα TextField και τον τύπο του ComboBox που θέλουμε.

- Η μέθοδος addCB ορίζει ένα ComboBox με τη βοήθεια της μεθόδου makeComboBoxes που θα περιγράψουμε σε λίγο. Αφού το ορίσει, το προσθέτει στο Panel, θέτει το text του TextField ίσο με "CB" και προσθέτει το ComboBox στο comboBoxList και το TextField στο textFieldList. Το TextField το χρησιμοποιούμε στη θέση του ComboBox για να χειριστούμε τα queries επειδή αρχικά δεν είχαμε ComboBoxes και οι μέθοδοι κατασκευής τους χρησιμοποιούν TextFields.
- Η μέθοδος makeComboBoxes ανάλογα με το String που δέχεται ως όρισμα συνδέεται με τη βάση και εκτελεί το αντίστοιχο SELECT βάζοντας τα αποτελέσματα σε ένα ComboBox το οποίο στη συνέχεια επιστρέφει.

Η addTFnLabels μετά το switch προσθέτει όποια TextFields χρειάζεται και ειδικά για τον πίνακα languages προσθέτει ένα Label το οποίο είναι ουσιαστικά το ENUM που έχει οριστεί στο CREATE της βάσης.

Ο πρώτος Listener είναι για ένα confirm κουμπί και το πρώτο πράγμα που κάνει είναι να καλέσει την μέθοδο allTextFieldsEmpty με όρισμα 'i' δηλαδή insert. Αν επιστρέψει true τότε πετάει OptionPane με ενημερωτικό μήνυμα.

- Η μέθοδος allTextFieldsEmpty διατρέχει το textFieldList και εάν βρει κάποιο που να μην είναι άδειο επιστρέφει false. Όπου υπάρχουν ComboBoxes και το action είναι 'i' η μέθοδος επιστρέφει false επειδή θα βρει κάποιο TextField που περιέχει το "CB". Αυτό δεν μας πειράζει στο insert γιατί τα ComboBoxes πάντα έχουν κάποια επιλογή και υπάρχουν πίνακες που έχουν μόνο ComboBoxes όπως ο πίνακας admins. Αντίθετα με action 'u' δηλαδή update μας

ενδιαφέρει να βλέπουμε τα TextFields για να καταλαβαίνουμε αν ο χρήστης έχει επιλέξει γραμμή για ενημέρωση, οπότε παραβλέπουμε τα TextFields με “CB”.

Αν επιστραφεί false τότε ο Listener καλεί την μέθοδο confirmation με όρισμα false και μόνο αν επιστραφεί true συνεχίζει. Ορίζει ένα array από Strings το οποίο γεμίζει με τη μέθοδο makeRow.

- Η μέθοδος makeRow διαβάζει τα περιεχόμενα των TextFields και τις επιλογές των ComboBoxes βάζοντάς τα σε ένα String array το οποίο και επιστρέφει.

Στη συνέχεια συνδέεται με τη βάση και καλεί την μέθοδο generateInsertQuery με ορίσματα το όνομα του πίνακα από το όρισμα του constructor και το array που φτιάχτηκε.

- Η μέθοδος generateInsertQuery με ένα StringBuilder δημιουργεί το INSERT query και το επιστρέφει ως String.

Μετά ο Listener εκτελεί το query και ενημερώνει και το JTable με τη νέα εγγραφή.

Ο δεύτερος Listener είναι για ένα Back κουμπί και όταν καλείται κάνει dispose το Insert και καλεί τον DBHandler.

Update

Η κλάση αυτή υιοθετεί την κλάση DBHandler και έχει ένα constructor, δύο ActionListeners, ένα MouseListener και τις μεθόδους:

- private String generateUpdateQuery(String tableName, String[] rowData, List<String> primaryKeyColumns)
- private boolean endsWithSubstring(StringBuilder stringBuilder, String substringToCheck)

Ο constructor ορίζει το βασικό Frame και τα χαρακτηριστικά του με τη βοήθεια ενός Form. Καλεί την diplayTableData και την addTFnLabels όπως και η Insert με μόνη διαφορά ότι η δεύερη παίρνει ως όρισμα action το ‘u’ δηλαδή update. Επειδή το action είναι διαφορετικό του ‘i’ η addTFnLabels προσθέτει TextFields και ComboBoxes για όλες τις στήλες του πίνακα.

Ο MouseListener ενεργοποιείται όταν ο χρήστης επιλέξει μία γραμμή του JTable και μεταφέρει τις τιμές στα αντίστοιχα TextFields παραλείποντας αυτά που έχουν “CB”.

Ο πρώτος Listener είναι για ένα confirm κουμπί και έχει παρόμοια δομή με αυτόν του Insert. Δηλαδή, αρχικά καλεί την allTextFieldEmpty και αν αυτή είναι true τότε πετάει κατάλληλο OptionPane. Διαφορετικά, καλεί την confirmation με όρισμα false και αν επιστρέψει true τότε συνεχίζει καλώντας την makeRow. Συνδέεται στη βάση και καλεί την getPrimaryKeyColumns η οποία περνάει ως όρισμα στη μέθοδο generateUpdateQuery μαζί με το αποτέλεσμα της makeRow και το όνομα του πίνακα από το όρισμα του constructor.

Η μέθοδος generateUpdateQuery χρησιμοποιώντας ένα StringBuilder κατασκευάζει το query. Αρχικά συγκρίνει το αποτέλεσμα του makeRow με αυτά που υπάρχουν στη γραμμή του JTable που έχει επιλεγεί και προσθέτει στο query μόνο αυτά που είναι διαφορετικά με τα σωστά datatypes. Αφού γίνει η σύγκριση καλείται η μέθοδος endsWithSubstring.

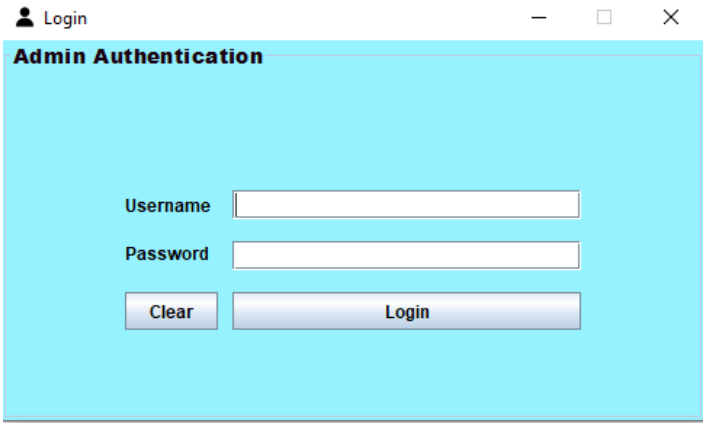
- Η μέθοδος endsWithSubstring παίρνει ως ορίσματα το StringBuilder και το String που θέλουμε να ελέγξουμε και επιστρέφει την ανάλογη boolean τιμή

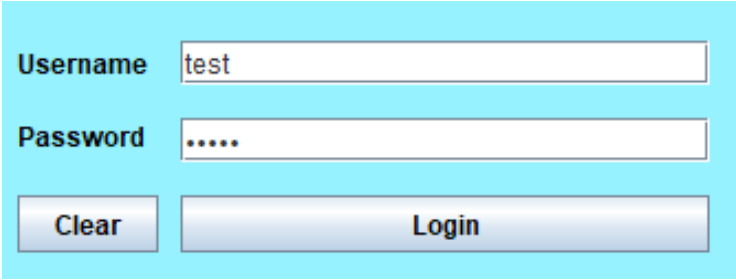
Αν ο StringBuilder τελειώνει με στο “ SET ” τότε το επιστρέφει ως String. Αν όχι τότε φτιάχνει και τοWHERE κομμάτι του query με τη βοήθεια της whereConditions και μετά το επιστρέφει.

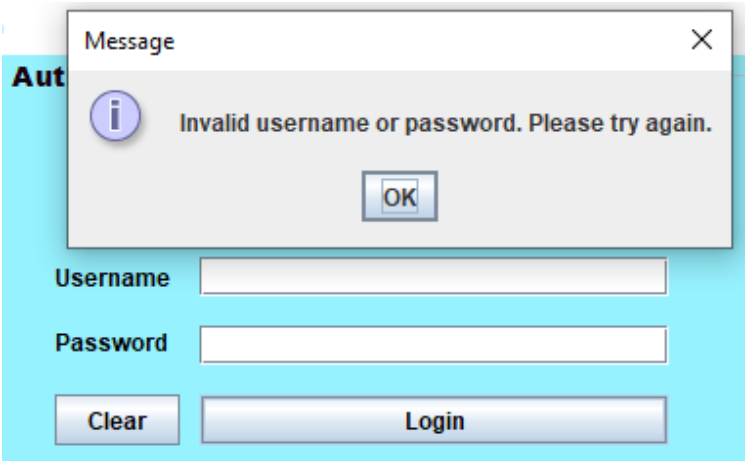
Όταν επιστρέψει η generateUpdateQuery ελέγχεται αν τελειώνει με το substring “ SET ” και αν ναι τότε πετάει OptionPane με κατάλληλο μήνυμα και σταματάει. Αλλιώς εκτελεί το query και αλλάζει και τη γραμμή στο JTable.

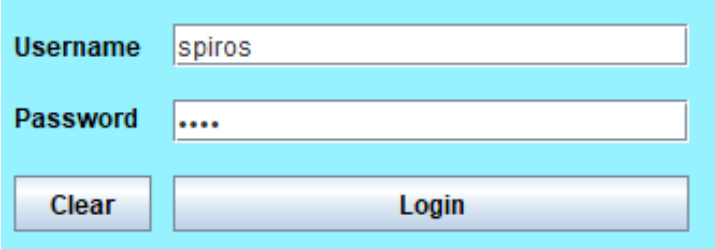
Ο δεύτερος Listener είναι για ένα Back κουμπί και είναι ίδιος με αυτόν του Insert.

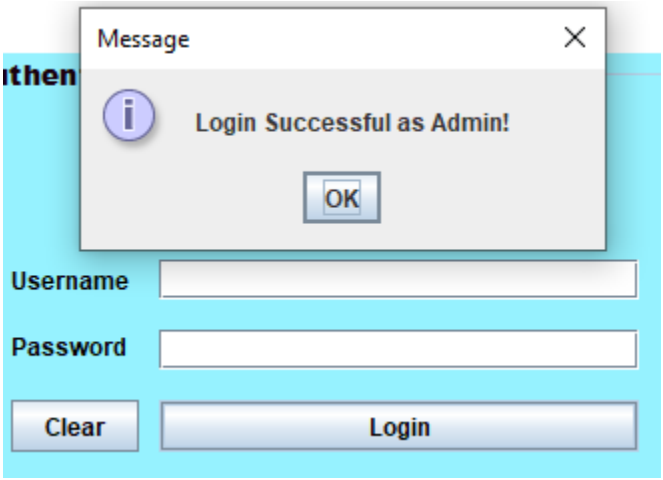
Screenshots από λειτουργία

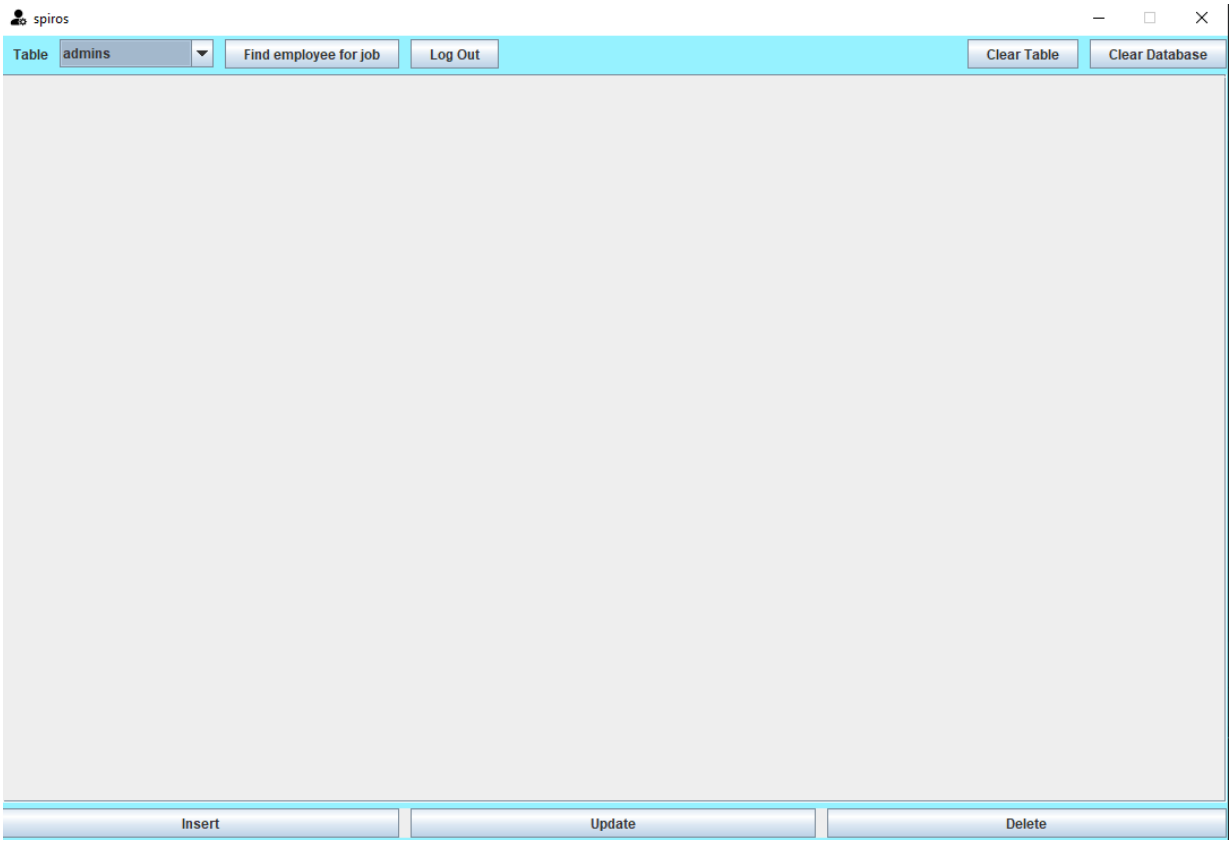


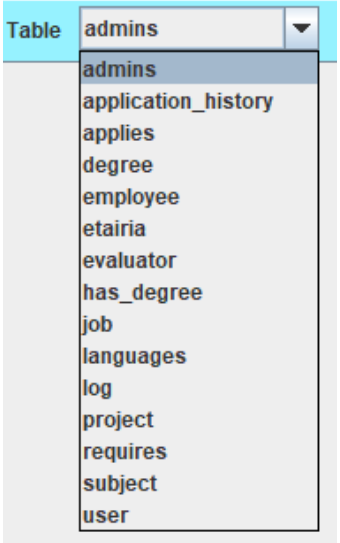


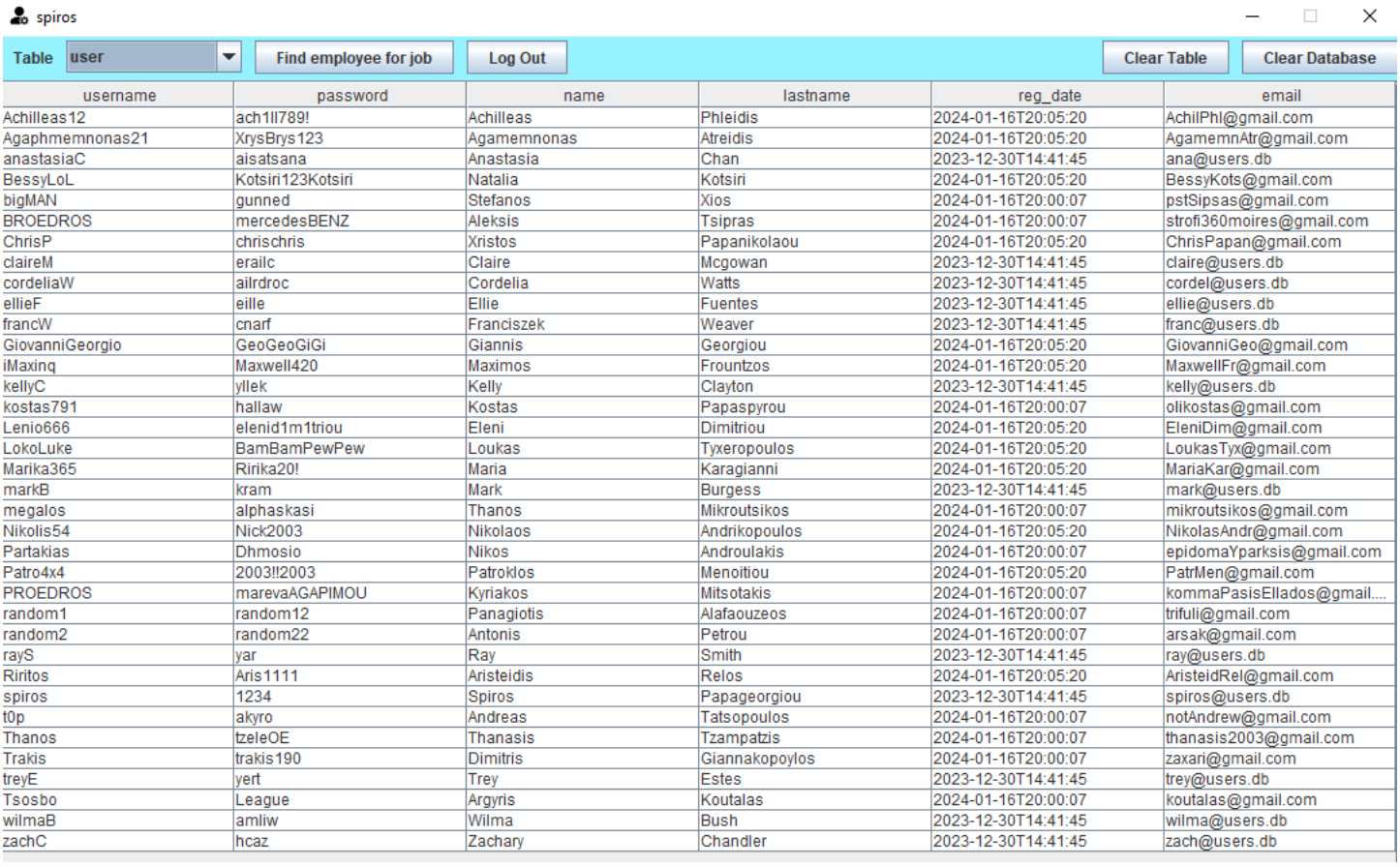



Username
spiros
Password
Clear
Login

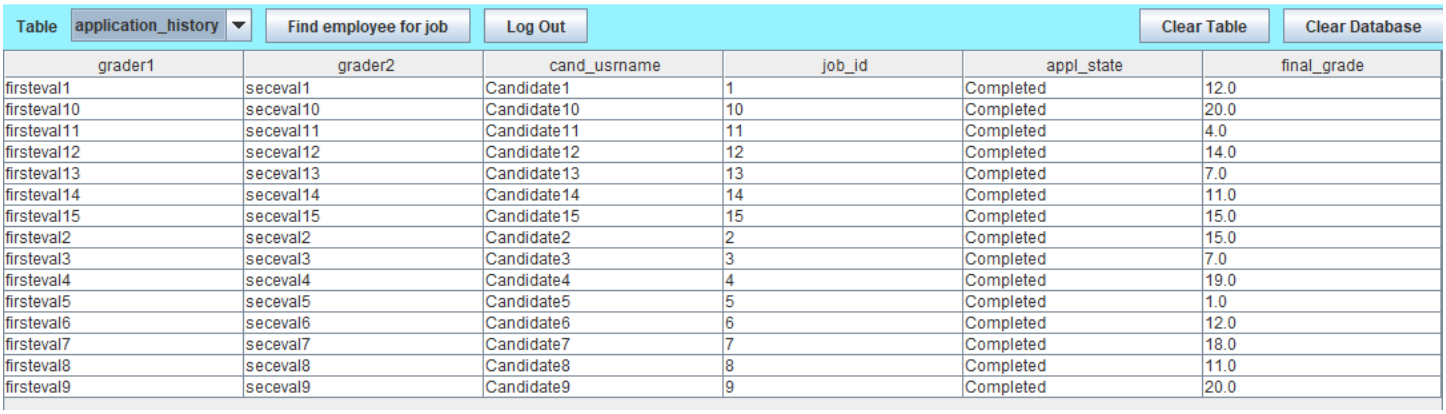

Message
Login Successful as Admin!
OK
Username
Password
Clear
Login


spiros
Table
admins
Find employee for job
Log Out
Clear Table
Clear Database
Insert
Update
Delete





username	password	name	lastname	reg_date	email
Achilleas12	ach1ll789!	Achilleas	Phleidis	2024-01-16T20:05:20	AchilPhl@gmail.com
Agaphmemnonas21	XrysBrys123	Agamemnonas	Atreidis	2024-01-16T20:05:20	AgamemnAtr@gmail.com
anastasiaC	aisatsana	Anastasia	Chan	2023-12-30T14:41:45	ana@users.db
BessyLoL	Kotsiri123Kotsiri	Natalia	Kotsiri	2024-01-16T20:05:20	BessyKots@gmail.com
bigMAN	gunned	Stefanos	Xios	2024-01-16T20:00:07	pstSipsas@gmail.com
BROEDROS	mercedesBENZ	Aleksis	Tsipras	2024-01-16T20:00:07	strofi360moires@gmail.com
ChrisP	chrischris	Xristos	Papanikolaou	2024-01-16T20:05:20	ChrisPapan@gmail.com
claireM	erailc	Claire	Mcgowan	2023-12-30T14:41:45	claire@users.db
cordeliaW	ailrdroc	Cordelia	Watts	2023-12-30T14:41:45	cordel@users.db
ellieF	eille	Ellie	Fuentes	2023-12-30T14:41:45	ellie@users.db
francW	cnarf	Franciszek	Weaver	2023-12-30T14:41:45	franc@users.db
GiovanniGeorgio	GeoGeoGiGi	Giannis	Georgiou	2024-01-16T20:05:20	GiovanniGeo@gmail.com
iMaxinq	Maxwell420	Maximos	Frountzos	2024-01-16T20:05:20	MaxwellFr@gmail.com
kellyC	yllek	Kelly	Clayton	2023-12-30T14:41:45	kelly@users.db
kostas791	hallaw	Kostas	Papaspyrou	2024-01-16T20:00:07	olikostas@gmail.com
Lenio666	elenid1m1triou	Eleni	Dimitriou	2024-01-16T20:05:20	EleniDim@gmail.com
LokoLuke	BamBamPewPew	Loukas	Tyxeropoulos	2024-01-16T20:05:20	LoukasTyx@gmail.com
Marika365	Ririka20!	Maria	Karagianni	2024-01-16T20:05:20	MariaKar@gmail.com
markB	kram	Mark	Burgess	2023-12-30T14:41:45	mark@users.db
megalos	alphaskasi	Thanos	Mikroutsikos	2024-01-16T20:00:07	mikroutsikos@gmail.com
Nikolis54	Nick2003	Nikolaos	Andrikopoulos	2024-01-16T20:05:20	NikolasAndr@gmail.com
Partakias	Dhmosio	Nikos	Androulakis	2024-01-16T20:00:07	epidomaYparksis@gmail.com
Patro4x4	2003!!2003	Patroklos	Menoitiou	2024-01-16T20:05:20	PatrMen@gmail.com
PROEDROS	marevaAGAPIMOU	Kyriakos	Mitsotakis	2024-01-16T20:00:07	kommaPasisEllados@gmail....
random1	random12	Panagiotis	Alafaouzeos	2024-01-16T20:00:07	trifuli@gmail.com
random2	random22	Antonis	Petrou	2024-01-16T20:00:07	arsak@gmail.com
rayS	yar	Ray	Smith	2023-12-30T14:41:45	ray@users.db
Riritos	Aris1111	Aristeidis	Relos	2024-01-16T20:05:20	AristeidRel@gmail.com
spiros	1234	Spiros	Papageorgiou	2023-12-30T14:41:45	spiros@users.db
t0p	akyro	Andreas	Tatsopoulos	2024-01-16T20:00:07	notAndrew@gmail.com
Thanos	tzeleOE	Thanasis	Tzampatzis	2024-01-16T20:00:07	thanasis2003@gmail.com
Trakis	trakis190	Dimitris	Giannakopoylos	2024-01-16T20:00:07	zaxari@gmail.com
treyE	yert	Trey	Estes	2023-12-30T14:41:45	trey@users.db
Tsosbo	League	Argyris	Koutalas	2024-01-16T20:00:07	koutalas@gmail.com
wilmaB	amliw	Wilma	Bush	2023-12-30T14:41:45	wilma@users.db
zachC	hcaz	Zachary	Chandler	2023-12-30T14:41:45	zach@users.db



grader1	grader2	cand_usrname	job_id	appl_state	final_grade
firsteval1	seceval1	Candidate1	1	Completed	12.0
firsteval10	seceval10	Candidate10	10	Completed	20.0
firsteval11	seceval11	Candidate11	11	Completed	4.0
firsteval12	seceval12	Candidate12	12	Completed	14.0
firsteval13	seceval13	Candidate13	13	Completed	7.0
firsteval14	seceval14	Candidate14	14	Completed	11.0
firsteval15	seceval15	Candidate15	15	Completed	15.0
firsteval2	seceval2	Candidate2	2	Completed	15.0
firsteval3	seceval3	Candidate3	3	Completed	7.0
firsteval4	seceval4	Candidate4	4	Completed	19.0
firsteval5	seceval5	Candidate5	5	Completed	1.0
firsteval6	seceval6	Candidate6	6	Completed	12.0
firsteval7	seceval7	Candidate7	7	Completed	18.0
firsteval8	seceval8	Candidate8	8	Completed	11.0
firsteval9	seceval9	Candidate9	9	Completed	20.0

Πατώντας Delete:

Table application_history | Find employee for job | Log Out | Clear Table | Clear Database

grader1	grader2	cand_usrname	job_id	appl_state	final_grade
firsteval1	seceval1	Candidate1	1	Completed	12.0
firsteval10	seceval10	Candidate10	10	Completed	20.0
firsteval11	seceval11	Candidate11	11	Completed	4.0
firsteval12	seceval12	Can		Completed	14.0
firsteval13	seceval13	Can		Completed	7.0
firsteval14	seceval14	Can		Completed	11.0
firsteval15	seceval15	Can		Completed	15.0
firsteval2	seceval2	Can		Completed	15.0
firsteval3	seceval3	Can		Completed	7.0
firsteval4	seceval4	Can		Completed	19.0
firsteval5	seceval5	Can		Completed	1.0
firsteval6	seceval6	Candidate6	6	Completed	12.0
firsteval7	seceval7	Candidate7	7	Completed	18.0
firsteval8	seceval8	Candidate8	8	Completed	11.0
firsteval9	seceval9	Candidate9	9	Completed	20.0

Message

Select a single row.

OK

Table application_history | Find employee for job | Log Out | Clear Table | Clear Database

grader1	grader2	cand_usrname	job_id	appl_state	final_grade
firsteval1	seceval1	Candidate1	1	Completed	12.0
firsteval10	seceval10	Candidate10	10	Completed	20.0
firsteval11	seceval11	Candidate11	11	Completed	4.0
firsteval12	seceval12	Candidate12	12	Completed	14.0
firsteval13	seceval13	Candidate13	13	Completed	7.0
firsteval14	seceval14	Candidate14	14	Completed	11.0
firsteval15	seceval15	Candidate15	15	Completed	15.0
firsteval2	seceval2	Can		Completed	15.0
firsteval3	seceval3	Can		Completed	7.0
firsteval4	seceval4	Can		Completed	19.0
firsteval5	seceval5	Can		Completed	1.0
firsteval6	seceval6	Can		Completed	12.0
firsteval7	seceval7	Can		Completed	18.0
firsteval8	seceval8	Can		Completed	11.0
firsteval9	seceval9	Can		Completed	20.0

Message

Select a single row.

OK

grader1	grader2	cand_usrname	job_id	appl_state	final_grade
firsteval1	seceval1	Candidate1	1	Completed	12.0
firsteval10	seceval10	Candidate10	10	Completed	20.0
firsteval11	seceval11			Completed	4.0
firsteval12	seceval12			Completed	14.0
firsteval13	seceval13			Completed	7.0
firsteval14	seceval14			Completed	11.0
firsteval15	seceval15			Completed	15.0
firsteval2	seceval2			Completed	15.0
firsteval3	seceval3			Completed	7.0
firsteval4	seceval4	Candidate4	4	Completed	19.0

Confirm

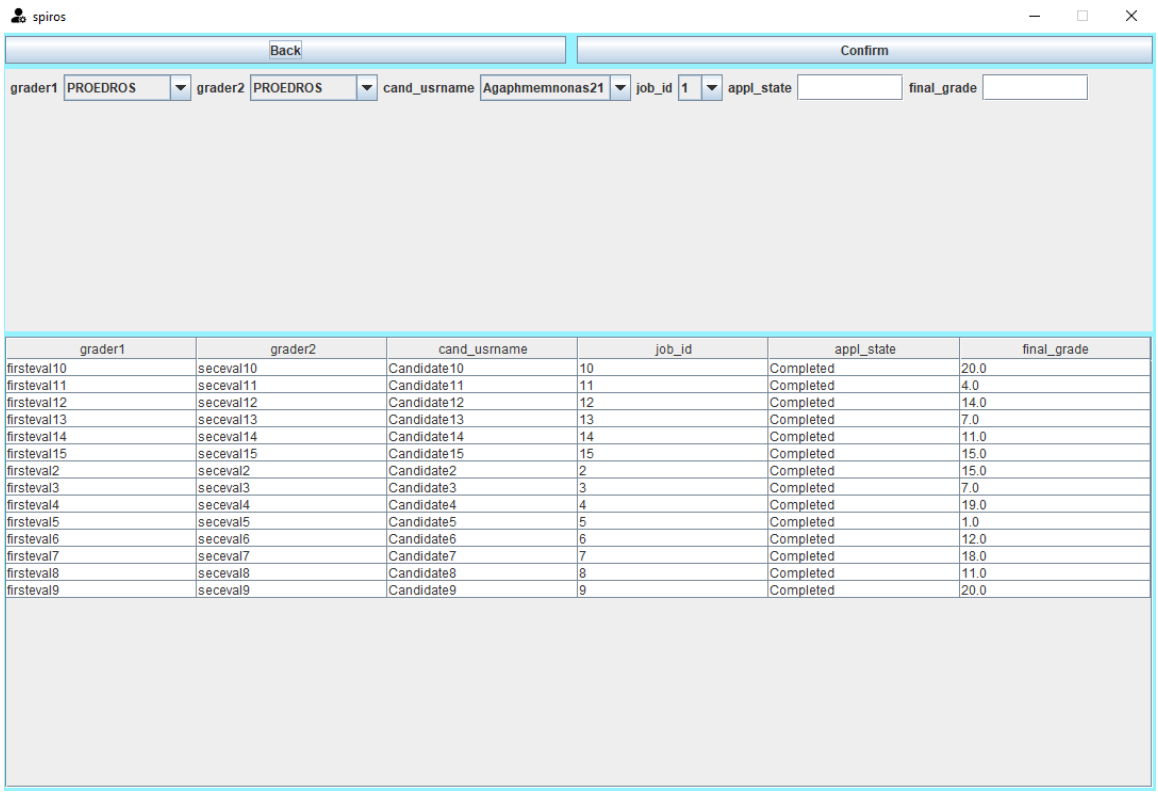
Are you sure you want to continue?

Yes | No

Με “No” ο πίνακας είναι ίδιος. Με “Yes”:

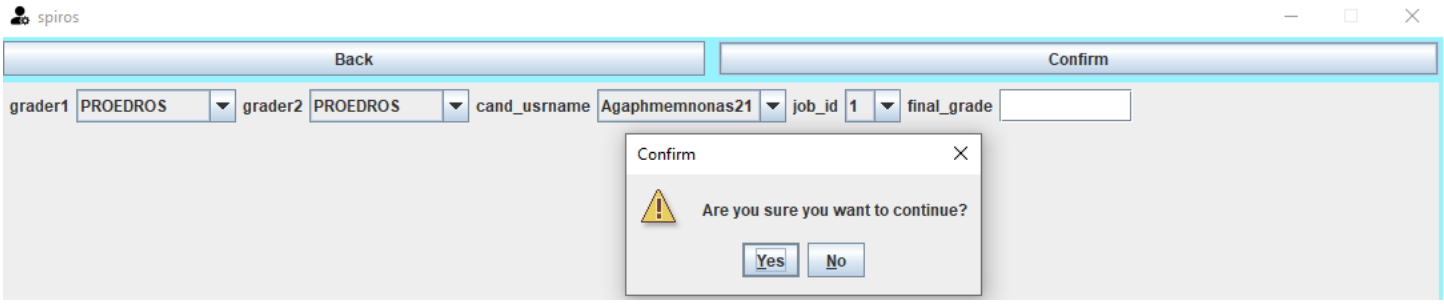
grader1	grader2	cand_usrname	job_id	appl_state	final_grade
firsteval10	seceval10	Candidate10	10	Completed	20.0
firsteval11	seceval11	Candidate11	11	Completed	4.0
firsteval12	seceval12	Candidate12	12	Completed	14.0
firsteval13	seceval13	Candidate13	13	Completed	7.0
firsteval14	seceval14	Candidate14	14	Completed	11.0
firsteval15	seceval15	Candidate15	15	Completed	15.0
firsteval2	seceval2	Candidate2	2	Completed	15.0
firsteval3	seceval3	Candidate3	3	Completed	7.0
firsteval4	seceval4	Candidate4	4	Completed	19.0
firsteval5	seceval5	Candidate5	5	Completed	1.0
firsteval6	seceval6	Candidate6	6	Completed	12.0
firsteval7	seceval7	Candidate7	7	Completed	18.0
firsteval8	seceval8	Candidate8	8	Completed	11.0
firsteval9	seceval9	Candidate9	9	Completed	20.0

Πατώντας Insert:

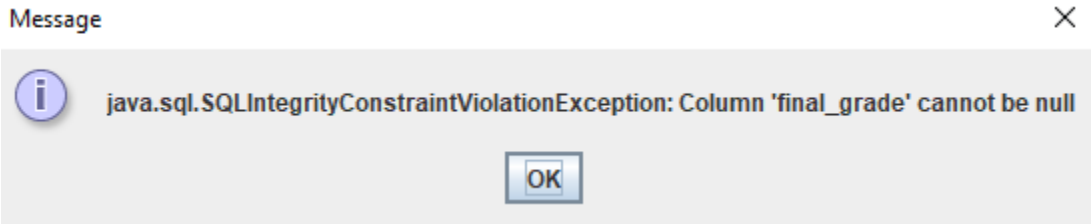


grader1	grader2	cand_usrname	job_id	appl_state	final_grade
firsteval10	seceval10	Candidate10	10	Completed	20.0
firsteval11	seceval11	Candidate11	11	Completed	4.0
firsteval12	seceval12	Candidate12	12	Completed	14.0
firsteval13	seceval13	Candidate13	13	Completed	7.0
firsteval14	seceval14	Candidate14	14	Completed	11.0
firsteval15	seceval15	Candidate15	15	Completed	15.0
firsteval2	seceval2	Candidate2	2	Completed	15.0
firsteval3	seceval3	Candidate3	3	Completed	7.0
firsteval4	seceval4	Candidate4	4	Completed	19.0
firsteval5	seceval5	Candidate5	5	Completed	1.0
firsteval6	seceval6	Candidate6	6	Completed	12.0
firsteval7	seceval7	Candidate7	7	Completed	18.0
firsteval8	seceval8	Candidate8	8	Completed	11.0
firsteval9	seceval9	Candidate9	9	Completed	20.0

Πατώντας Confirm:

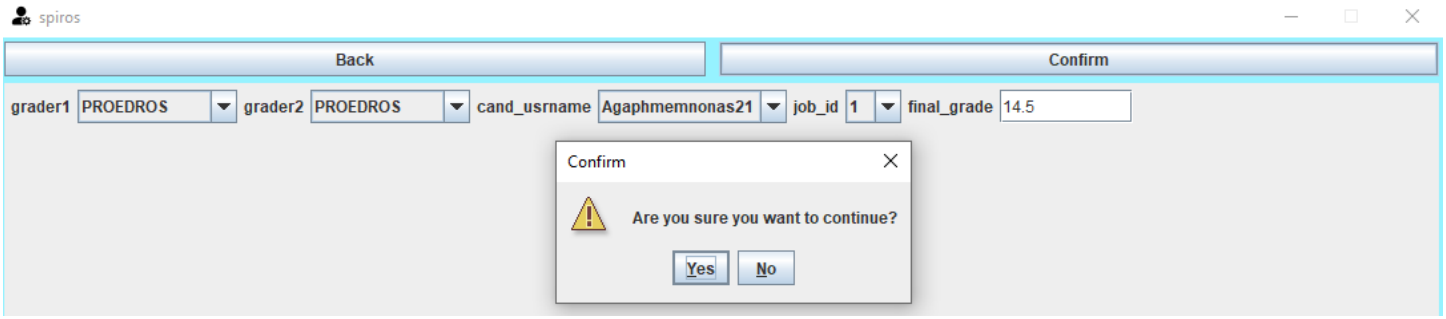


Με “Yes”:



Και ο πίνακας δεν άλλαξε.

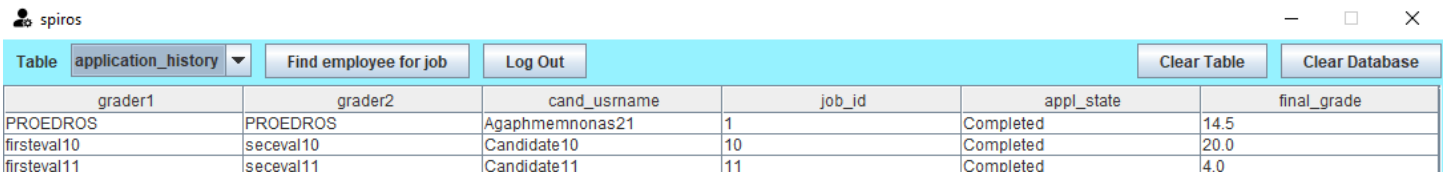
Πατώντας Confirm και πάλι:



Με “Yes”:

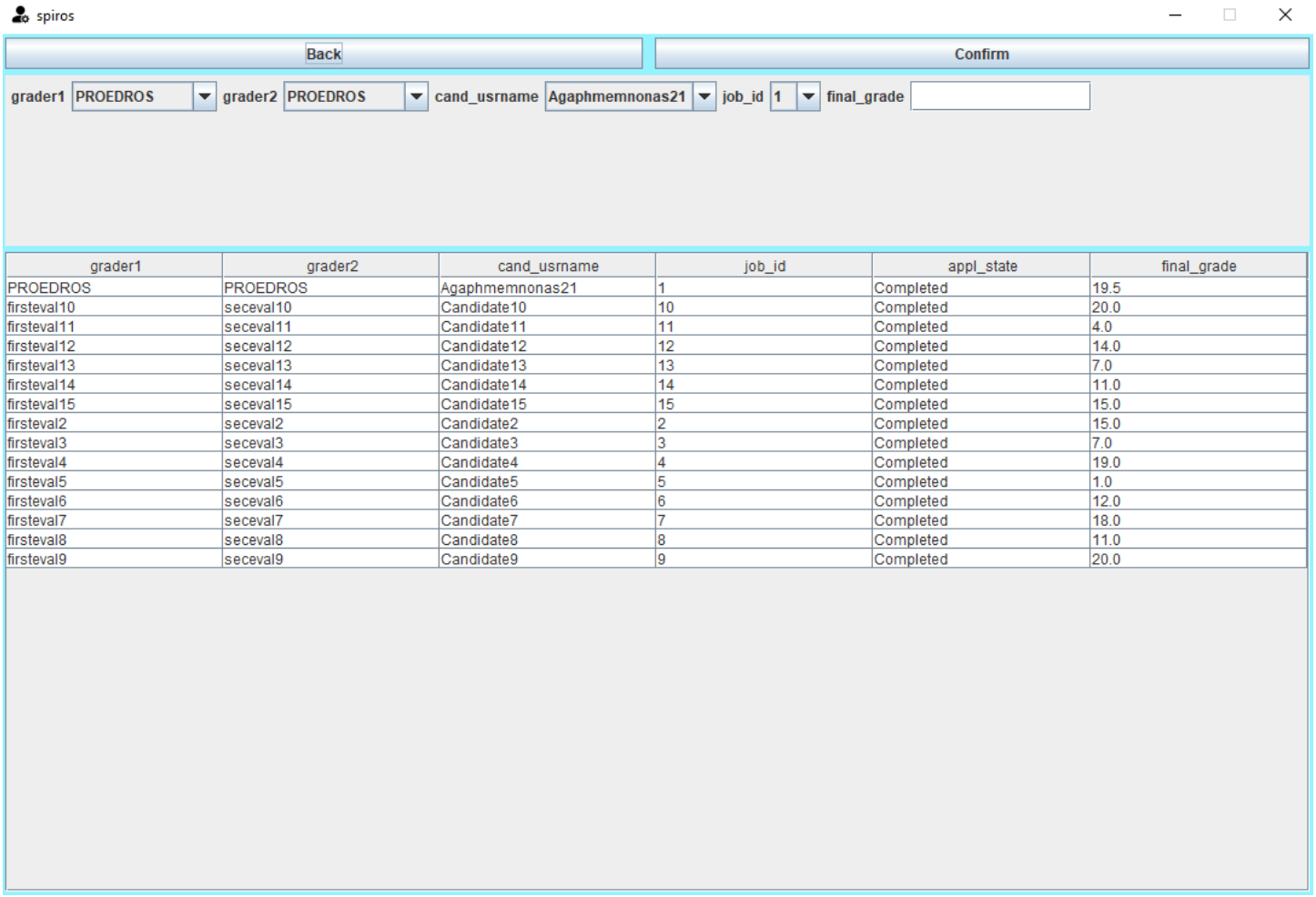
grader1	grader2	cand_usrname	job_id	appl_state	final_grade
firsteval10	seceval10	Candidate10	10	Completed	20.0
firsteval11	seceval11	Candidate11	11	Completed	4.0
firsteval12	seceval12	Candidate12	12	Completed	14.0
firsteval13	seceval13	Candidate13	13	Completed	7.0
firsteval14	seceval14	Candidate14	14	Completed	11.0
firsteval15	seceval15	Candidate15	15	Completed	15.0
firsteval2	seceval2	Candidate2	2	Completed	15.0
firsteval3	seceval3	Candidate3	3	Completed	7.0
firsteval4	seceval4	Candidate4	4	Completed	19.0
firsteval5	seceval5	Candidate5	5	Completed	1.0
firsteval6	seceval6	Candidate6	6	Completed	12.0
firsteval7	seceval7	Candidate7	7	Completed	18.0
firsteval8	seceval8	Candidate8	8	Completed	11.0
firsteval9	seceval9	Candidate9	9	Completed	20.0
PROEDROS	PROEDROS	Agaphmemnonas21	1	14.5	

Το αποτέλεσμα απλά φαίνεται λάθος. Αν πατήσουμε back και ξανά επιλέξουμε τον πίνακα:



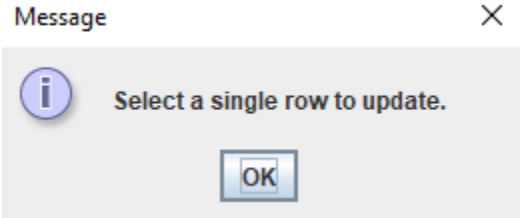
grader1	grader2	cand_usrname	job_id	appl_state	final_grade
PROEDROS	PROEDROS	Agaphmemnonas21	1	Completed	14.5
firsteval10	seceval10	Candidate10	10	Completed	20.0
firsteval11	seceval11	Candidate11	11	Completed	4.0

Πατώντας Update:

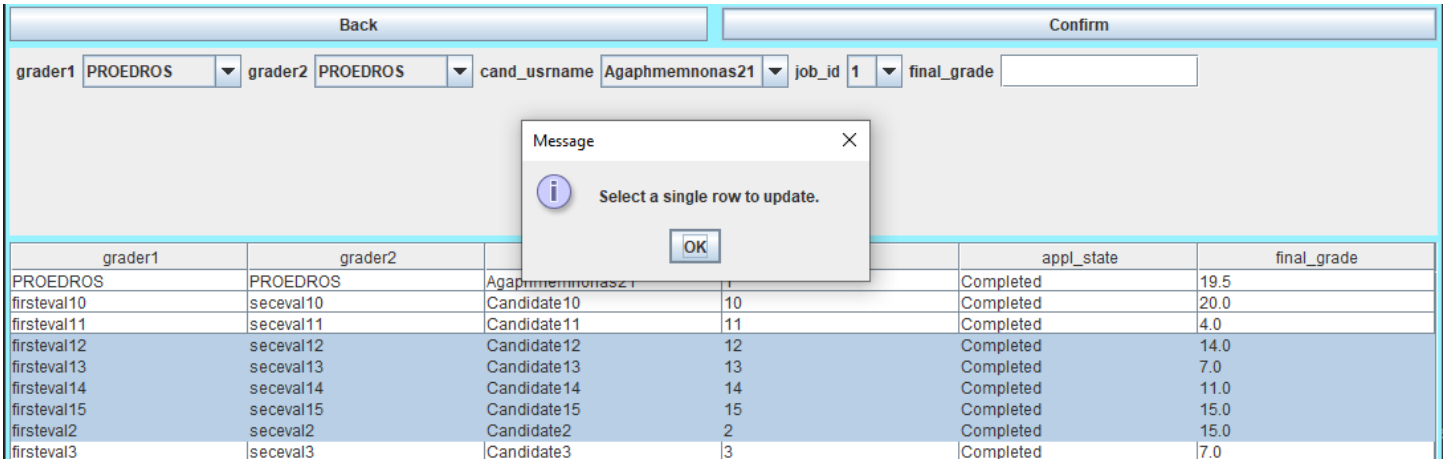


grader1	grader2	cand_usrname	job_id	appl_state	final_grade
PROEDROS	PROEDROS	Agaphmemnonas21	1	Completed	19.5
firsteval10	seceval10	Candidate10	10	Completed	20.0
firsteval11	seceval11	Candidate11	11	Completed	4.0
firsteval12	seceval12	Candidate12	12	Completed	14.0
firsteval13	seceval13	Candidate13	13	Completed	7.0
firsteval14	seceval14	Candidate14	14	Completed	11.0
firsteval15	seceval15	Candidate15	15	Completed	15.0
firsteval2	seceval2	Candidate2	2	Completed	15.0
firsteval3	seceval3	Candidate3	3	Completed	7.0
firsteval4	seceval4	Candidate4	4	Completed	19.0
firsteval5	seceval5	Candidate5	5	Completed	1.0
firsteval6	seceval6	Candidate6	6	Completed	12.0
firsteval7	seceval7	Candidate7	7	Completed	18.0
firsteval8	seceval8	Candidate8	8	Completed	11.0
firsteval9	seceval9	Candidate9	9	Completed	20.0

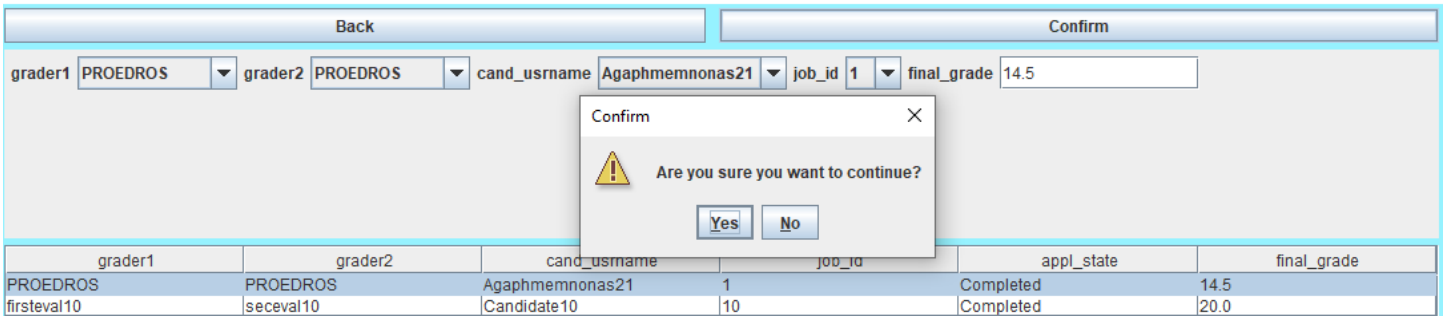
Πατώντας Confirm:



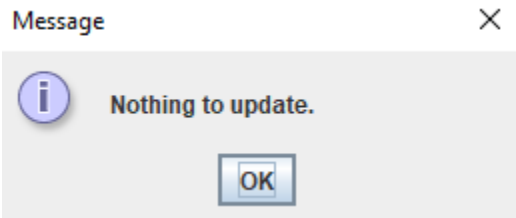
Πατώντας πάλι Confirm:



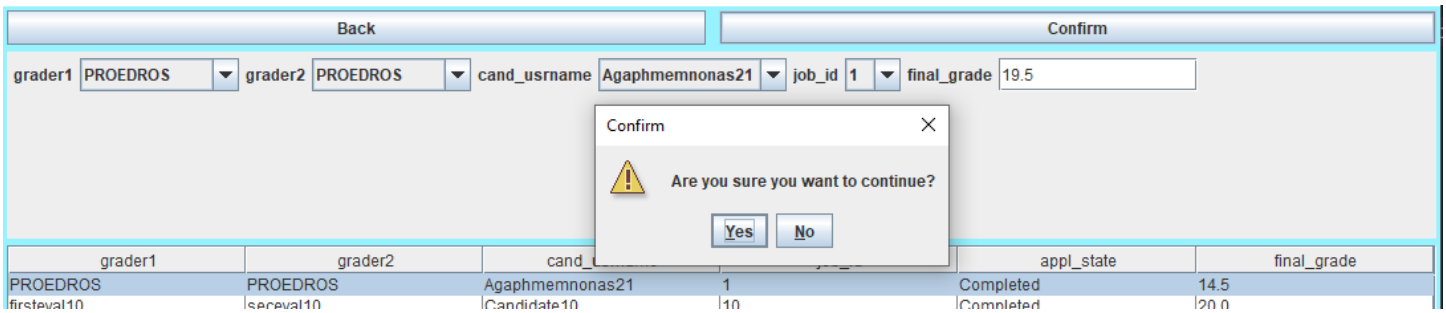
Πατώντας πάλι Confirm:



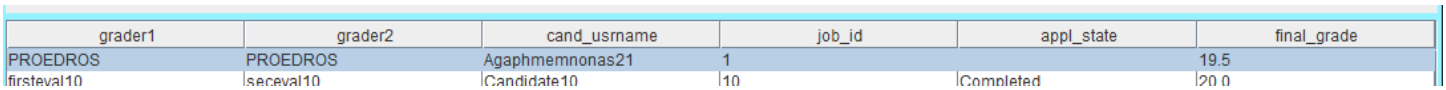
Με “Yes”:



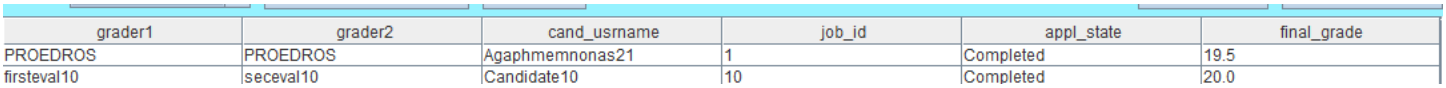
Πατώντας πάλι Confirm:



Με “Yes”:



Το αποτέλεσμα απλά φαίνεται λάθος. Αν πατήσουμε back και ξανά επιλέξουμε τον πίνακα:



Κεφάλαιο 5

Ως μπόνους έχουμε κάνει τα εξής. Η διεπαφή μπορεί να χρησιμοποιηθεί και από εργαζομένους. Στην κλάση login η μέθοδος credentialsCheck, αν το SELECT από τον πίνακα admins δεν επιστρέψει κάτι τότε δοκιμάζει άλλο ένα SELECT στον πίνακα employee και αν βρεθεί αποτέλεσμα τότε επιστρέφει τον χαρακτήρα ‘e’. Το switch του constructor αν λάβει ‘e’ κάνει dispose και καλεί την κλάση userHandler.

Για την περίπτωση του admin στη κλάση DBHandler ο χρήστης έχει την επιλογή να κάνει Clear ένα πίνακα ή όλη τη βάση, να κάνει Logout και να τρέξει τη stored procedure του ερωτήματος 3.1.3.3.

Για το Clear υπάρχουν δύο κουμπιά που συνδέονται στη βάση. Το ένα καλεί μια stored procedure που με όρισμα το όνομα ενός πίνακα εκτελεί ένα DELETE query. Το δεύτερο καλεί μια άλλη stored procedure η οποία χρησιμοποιεί την πρώτη για να κάνει DELETE όλα τα δεδομένα. Και στις δύο περιπτώσεις καλείται η confirmation δύο φορές, μία με όρισμα false και μία με true.

Το Logout κάνει dispose το DBHandler και καλεί την login.

Η stored procedure του 3.1.3.3 καλείται από τον Listener ενός κουμπιού αφού πρώτα αυτός έχει καλέσει την μέθοδο public static int jobOptionPane().

- Η μέθοδος jobOptionPane ορίζει ένα ComboBox και το γεμίζει με τη μέθοδο makeComboBoxes με όρισμα "jobs". Στη συνέχεια πετάει ένα OptionPane που ο χρήστης καλείται να επιλέξει job id για το οποίο θέλει να βρει τον κατάλληλο εργαζόμενο. Επιστρέφει την επιλογή αυτή.

Μετά την κλήση της SP εμφανίζεται ενημερωτικό μήνυμα με OptionPane που λέει αν και ποιος πήρε τη δουλειά.

Στην περίπτωση που κάνει login κάποιος employee καλείται η κλάση userHandler. Η κλάση αυτή υιοθετεί την κλάση JFrame και έχει ένα constructor, επτά ActionListeners και την μέθοδο private void applySP(String candidate, char action). Ο constructor ορίζει το βασικό Frame και τα χαρακτηριστικά του με τη βοήθεια ενός Form.

Υπάρχουν τρία κουμπιά προκειμένου ο χρήστης να δει κάποιον πίνακα της βάσης. Οι πίνακες που μπορεί να δει είναι job, application_history, applies. Και τα τρία καλούν την displayTableData του DBHandler με ορίσματα το όνομα του JTable στο οποίο θα τυπωθούν τα δεδομένα, το όνομα του πίνακα της βάσης, το char 'u' επειδή δεν είναι admins και το όνομά με το οποίο έκαναν login.

Η displayTableData αν το char είναι 'u' και ανάλογα με το όνομα του πίνακα τυπώνει μόνο όσα πεδία χρειάζονται. Για παράδειγμα στο job δεν βλέπει τους βαθμολογητές και στους άλλους δύο πίνακες βλέπει μόνο το δικό του ιστορικό και μόνο τις δικές του αιτήσεις.

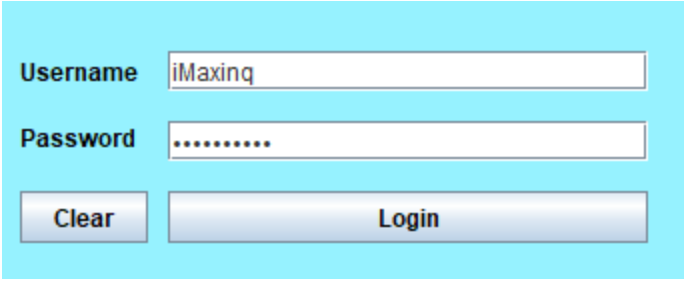
Υπάρχει επίσης ένα κουμπί Logout που είναι ίδιο με αυτό της κλάσης DBHandler.

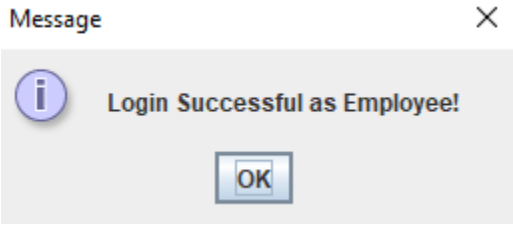
Υπάρχουν τρία κουμπιά προκειμένου ο χρήστης να μπορεί να υποβάλει, να ακυρώσει ή να ενεργοποιήσει κάποια αίτηση για μία θέση εργασίας. Τα κουμπιά αυτά καλούν και τα τρία την applySP με ορίσματα το όνομα του χρήστη και το αντίστοιχο char (i,c,a).

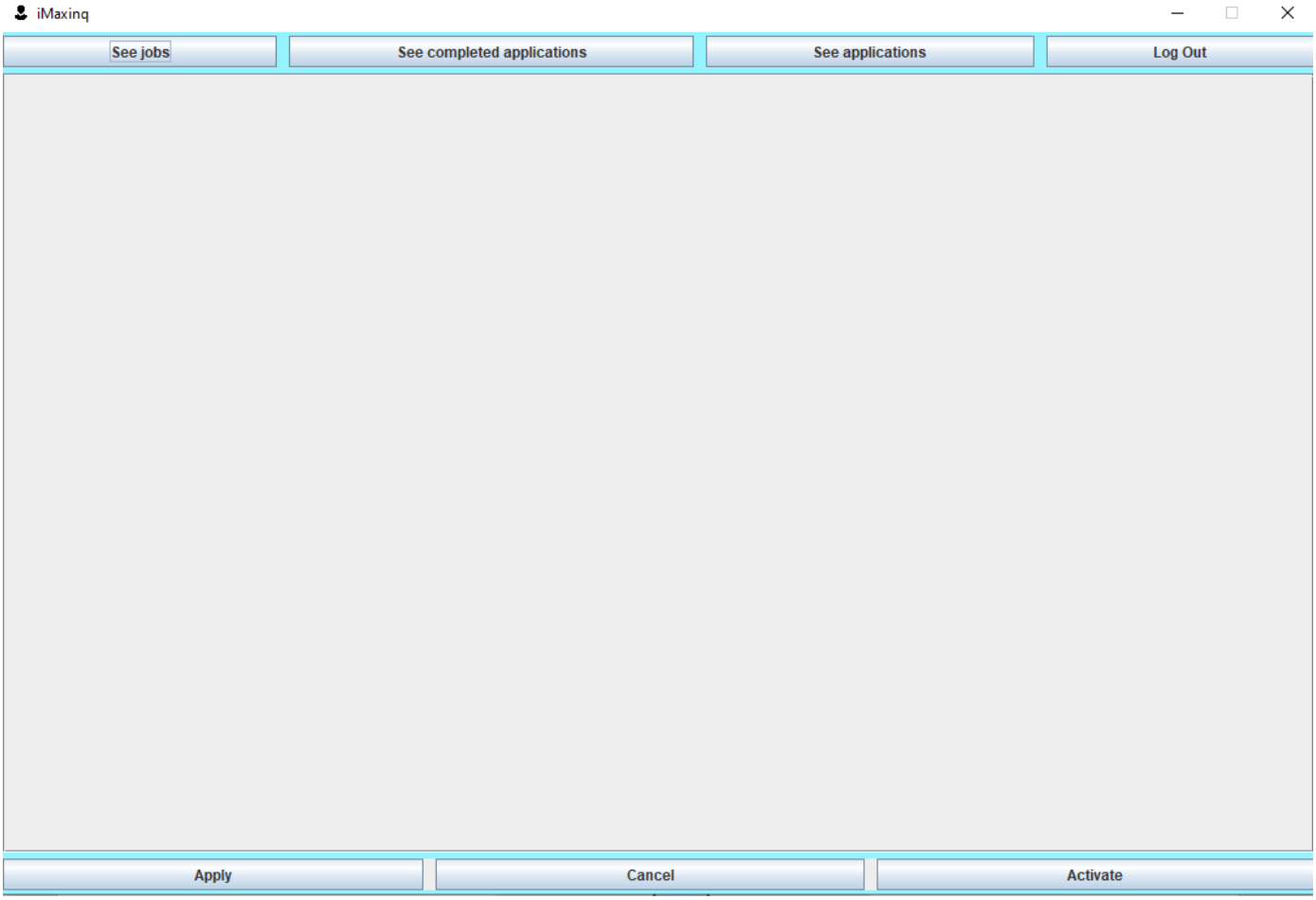
Η μέθοδος applySP συνδέεται με τη βάση και καλεί την stored procedure του ερωτήματος 3.1.3.2 αφού πρώτα καλέσει την jobOptionPane και την confirmation.

Ακόμα κάθε χρήστης μπορεί να ταξινομήσει τα δεδομένα του JTable που βλέπει κάθε φορά.

Screenshots από λειτουργία







Πατώντας See jobs:

See jobs | See completed applications | See applications | Log Out

id	start_date	salary	position	edra	announce_date	submission_date	cancellation_deadline
1	2024-02-01	3500.0	Manager	Athens	2023-12-30T14:41:45	2024-01-17	2024-01-22
2	2024-02-03	2200.0	Researcher	Athens	2023-12-30T14:41:45	2024-01-19	2024-01-24
3	2024-01-15	1400.0	Intermediate Software ...	Thesaloniki	2023-12-30T14:41:45	2023-12-31	2024-01-05
4	2024-01-14	1500.0	Intermediate Software ...	Thesaloniki	2023-12-30T14:41:45	2023-12-30	2024-01-04
5	2024-02-18	1400.0	Junior Software Engin...	Patra	2023-12-30T14:41:45	2024-02-03	2024-02-08
6	2024-04-23	1200.0	Junior Software Engin...	Athens	2023-12-30T14:41:45	2024-04-08	2024-04-13
7	2024-04-01	2700.0	Assistant Attorney	Athens	2023-12-30T14:41:45	2024-03-17	2024-03-22
8	2024-06-01	1000.0	Internship	Patra	2023-12-30T14:41:45	2024-05-17	2024-05-22
11	2024-01-05	500.0	Clerk	Athens	2023-12-31T14:23:16	2023-12-21	2023-12-26
12	2024-04-12	2300.0	Mousikos se sxoleio	Patra	2024-01-16T20:00:07	2024-03-28	2024-04-02
13	2024-05-11	2100.0	Filologos se sxoleio	Patra	2024-01-16T20:00:07	2024-04-26	2024-05-01
14	2024-06-09	4100.0	Kathigitis Kryptografias	Athens	2024-01-16T20:00:07	2024-05-25	2024-05-30
15	2024-07-28	700.0	Mousikos se sxoleio	Thesaloniki	2024-01-16T20:00:07	2024-07-13	2024-07-18
16	2024-08-29	1000.0	Analytis keimenwn	Patra	2024-01-16T20:00:07	2024-08-14	2024-08-19
17	2024-10-12	800.0	Fysikos se sxoleio	Athens	2024-01-16T20:00:07	2024-09-27	2024-10-02
18	2024-11-07	750.0	Fysikos se frontistirio	Patra	2024-01-16T20:00:07	2024-10-23	2024-10-28
19	2024-08-16	950.0	Cybersecutiry engineer	Thesaloniki	2024-01-16T20:00:07	2024-08-01	2024-08-06
20	2024-02-01	3500.0	Manager	Athens	2024-01-16T20:05:20	2024-01-17	2024-01-22
21	2024-02-03	3100.0	Mathematician	Patras	2024-01-16T20:05:20	2024-01-19	2024-01-24
22	2024-09-15	1300.0	Software Engineer	Patras	2024-01-16T20:05:20	2024-08-31	2024-09-05
23	2024-01-14	1600.0	Software Engineer	Thessaloniki	2024-01-16T20:05:20	2023-12-30	2024-01-04
24	2024-11-18	1100.0	Accountant	Patras	2024-01-16T20:05:20	2024-11-03	2024-11-08
25	2024-04-23	1100.0	Software Engineer	Thessaloniki	2024-01-16T20:05:20	2024-04-08	2024-04-13
26	2024-04-01	2760.0	Software Engineer	Athens	2024-01-16T20:05:20	2024-03-17	2024-03-22
27	2024-12-01	2100.0	Civil Engineer	Patras	2024-01-16T20:05:20	2024-11-16	2024-11-21

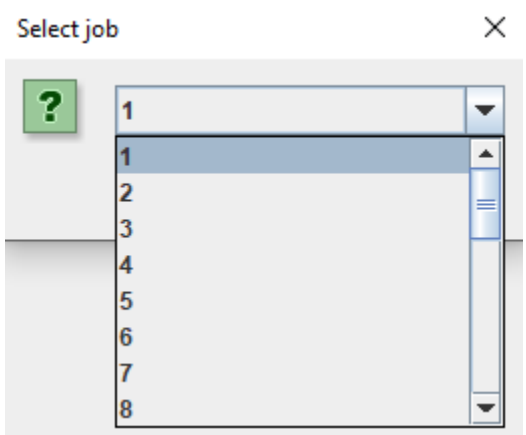
Πατώντας See completed applications:

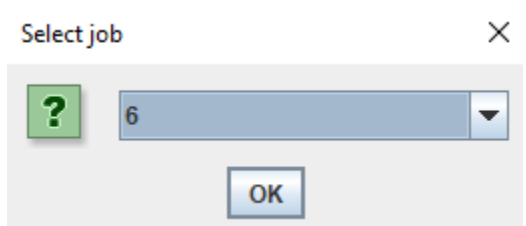
job_id	appl_state	final_grade

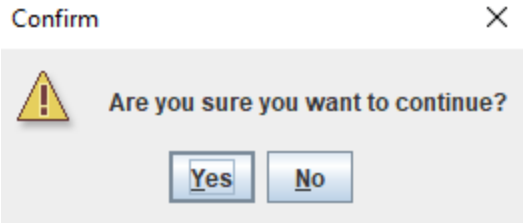
Πατώντας See applications:

job_id	appl_state	appl_date	grade1	grade2
1	Active	2024-01-17T12:58:37		
22	Cancelled	2024-01-16T20:16:48		
26	Active	2024-01-16T20:16:48		

Πατώντας Apply:



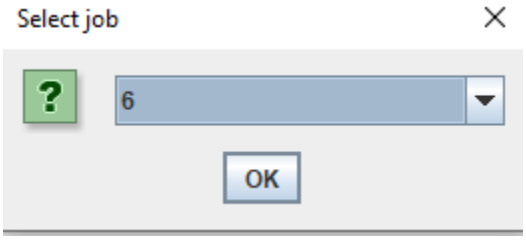


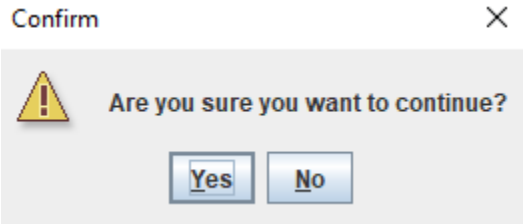


Με “Yes”:

job_id	appl_state	appl_date	grade1	grade2
1	Active	2024-01-17T12:58:37		
6	Active	2024-01-19T19:47:56		
22	Cancelled	2024-01-16T20:16:48		
26	Active	2024-01-16T20:16:48		

Πατώντας Cancel:

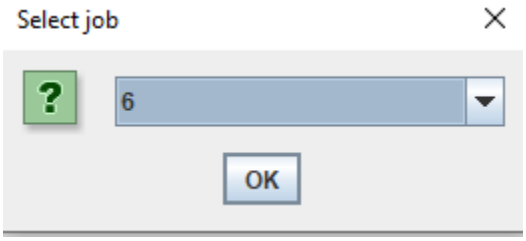


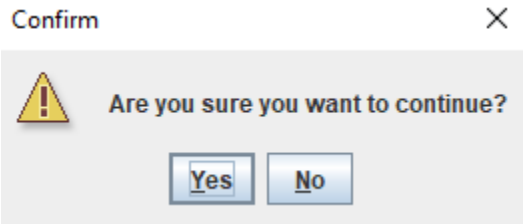


Με “Yes”:

job_id	appl_state	appl_date	grade1	grade2
1	Active	2024-01-17T12:58:37		
6	Cancelled	2024-01-19T19:47:56		
22	Cancelled	2024-01-16T20:16:48		
26	Active	2024-01-16T20:16:48		

Πατώντας Activate:





Με “Yes”:

job_id	appl_state	appl_date	grade1	grade2
1	Active	2024-01-17T12:58:37		
6	Active	2024-01-19T19:47:56		
22	Cancelled	2024-01-16T20:16:48		
26	Active	2024-01-16T20:16:48		

Για να δούμε τα ενημερωμένα δεδομένα πρέπει να ξαναπατήσουμε το See applications.

Κάνουμε Logout και μπαίνουμε ως admin:

Ταξινομούμε τον πίνακα applies με βάση το appl_date:

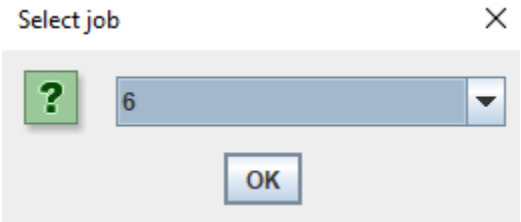
spiros

Table applies | Find employee for job | Log Out | Clear Table | Clear Database

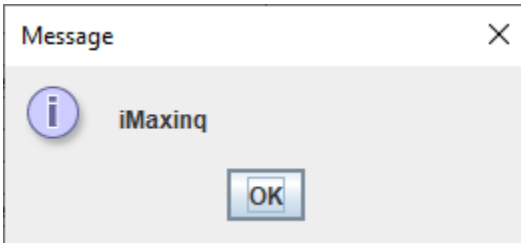
cand_usrname	job_id	appl_state	appl_date	grade1	grade2
iMaxinq	6	Active	2024-01-19T19:47:56		
iMaxinq	1	Active	2024-01-17T12:58:37		
Agaphmemnonas21	27	Cancelled	2024-01-16T20:16:48		
ChrisP	26	Active	2024-01-16T20:16:48		

Βλέπουμε ότι η εγγραφή από τον employee έχει μπει.

Πατάμε find employee for job:



Με αποτέλεσμα:



Από τον πίνακα applies χάθηκαν όλες οι εγγραφές με job_id ίσο με 6. Έχουν μεταφερθεί στο application_history.

spiros

Table application_history | Find employee for job | Log Out | Clear Table | Clear Database

grader1	grader2	cand_usrname	job_id	appl_state	final_grade
GiovanniGeorgio	Lenio666	iMaxinq	6	Completed	6.0
firsteval9	seceval9	Candidate9	9	Completed	20.0
firsteval8	seceval8	Candidate8	8	Completed	11.0
firsteval7	seceval7	Candidate7	7	Completed	18.0
firsteval6	seceval6	Candidate6	6	Completed	12.0
firsteval5	seceval5	Candidate5	5	Completed	1.0
firsteval4	seceval4	Candidate4	4	Completed	19.0
firsteval3	seceval3	Candidate3	3	Completed	7.0
firsteval2	seceval2	Candidate2	2	Completed	15.0
firsteval15	seceval15	Candidate15	15	Completed	15.0
firsteval14	seceval14	Candidate14	14	Completed	11.0
firsteval13	seceval13	Candidate13	13	Completed	7.0
firsteval12	seceval12	Candidate12	12	Completed	14.0
firsteval11	seceval11	Candidate11	11	Completed	4.0
firsteval10	seceval10	Candidate10	10	Completed	20.0
PROEDROS	PROEDROS	Agaphmemnonas21	1	Completed	19.5

spiros

Table: applies | Find employee for job | Log Out | Clear Table | Clear Database

cand_usrname	job_id	appl_state	appl_date	grade1	grade2
iMaxinq	1	Active	2024-01-17T12:58:37		
spiros	1	Active	2023-12-30T14:41:45		
wilmaB	2	Active	2023-12-30T14:41:45		
spiros	3	Cancelled	2023-12-30T14:41:45		
spiros	4	Active	2023-12-30T14:41:45		
wilmaB	5	Cancelled	2023-12-30T14:41:45		
cordeliaW	8	Active	2023-12-30T14:41:45		
markB	11	Active	2023-12-31T14:41:28		
megalos	12	Cancelled	2024-01-16T20:03:21		
Partakias	12	Active	2024-01-16T20:03:21		
Thanos	13	Active	2024-01-16T20:03:21		
BROEDROS	14	Active	2024-01-16T20:03:21		
megalos	15	Cancelled	2024-01-16T20:03:21		
bigMAN	16	Active	2024-01-16T20:03:21		
Thanos	17	Cancelled	2024-01-16T20:03:21		
Partakias	18	Active	2024-01-16T20:03:21		
BROEDROS	19	Active	2024-01-16T20:03:21		
Marika365	20	Active	2024-01-16T20:16:48		
iMaxinq	22	Cancelled	2024-01-16T20:16:48		
LokoLuke	24	Cancelled	2024-01-16T20:16:48		
ChrisP	26	Active	2024-01-16T20:16:48		
iMaxinq	26	Active	2024-01-16T20:16:48		
Agaphmemnonas21	27	Cancelled	2024-01-16T20:16:48		

Ξανακάνουμε Login ως employee:

Ιστορικό:

iMaxinq

See jobs | See completed applications | See applications | Log Out

job_id	appl_state	final_grade
6	Completed	6.0

Αιτήσεις:

iMaxinq

See jobs | See completed applications | See applications | Log Out

job_id	appl_state	appl_date	grade1	grade2
1	Active	2024-01-17T12:58:37		
22	Cancelled	2024-01-16T20:16:48		
26	Active	2024-01-16T20:16:48		

Τώρα θα συνδεθούμε ως admin και θα κάνουμε Cleat Table:

spiros

Table: application_history | Find employee for job | Log Out | Clear Table | Clear Database

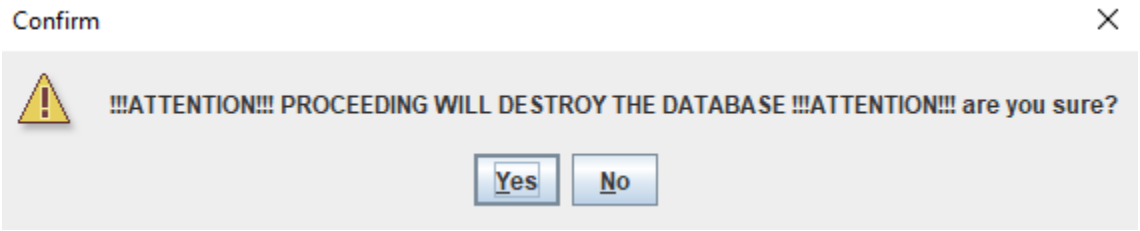
grader1	grader2	cand_usrname	job_id	appl_state	final_grade
PROEDROS	PROEDROS	Agaphmemnonas21	1	Completed	19.5
firsteval10	seceval10	Candidate10	10	Completed	20.0
firsteval11	seceval11	Candidate11	11	Completed	4.0
firsteval12	seceval12	Candi		Completed	14.0
firsteval13	seceval13	Candi		Completed	7.0
firsteval14	seceval14	Candi		Completed	11.0
firsteval15	seceval15	Candi		Completed	15.0
firsteval2	seceval2	Candi		Completed	15.0
firsteval3	seceval3	Candi		Completed	7.0
firsteval4	seceval4	Candi		Completed	19.0
firsteval5	seceval5	Candi		Completed	1.0
firsteval6	seceval6	Candidate6	6	Completed	12.0
firsteval7	seceval7	Candidate7	7	Completed	18.0
firsteval8	seceval8	Candidate8	8	Completed	11.0
firsteval9	seceval9	Candidate9	9	Completed	20.0
GiovanniGeorgio	Lenio666	iMaxinq	6	Completed	6.0

Confirm

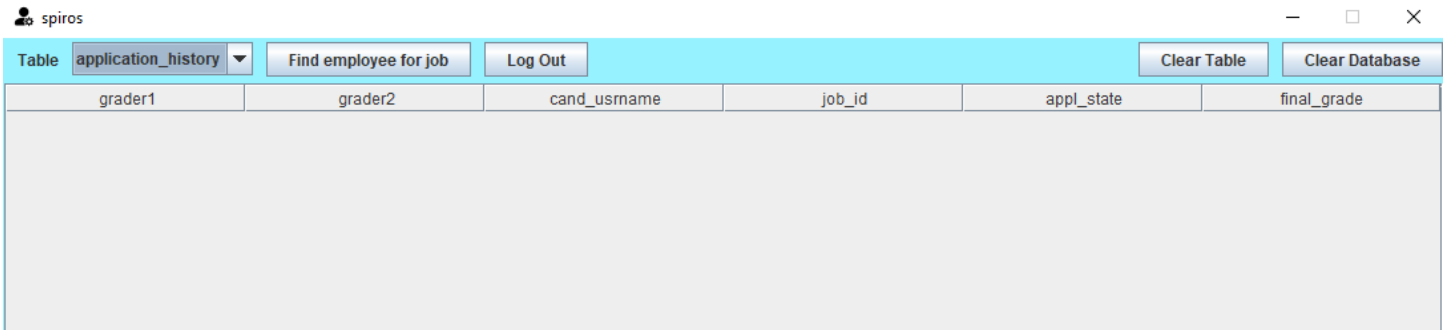
Are you sure you want to continue?

Yes | No

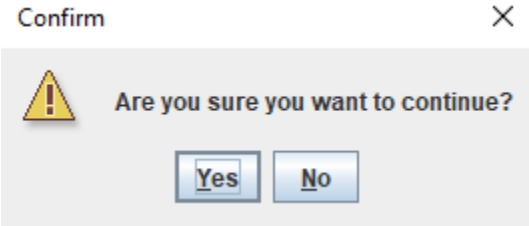
Με “Yes”:



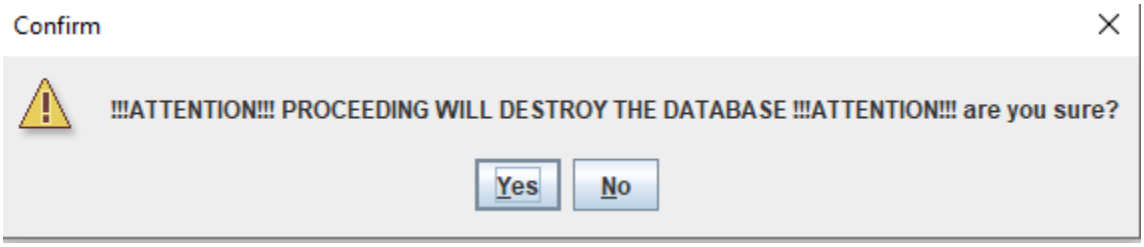
Με “Yes”:



Τώρα θα κάνουμε Cleat Database:



Με “Yes”:



Με “Yes” έχουν διαγραφεί όλα τα δεδομένα από τη βάση εκτός από τον πίνακα log:

spiros

Table log | Find employee for job | Log Out | Clear Table | Clear Database

id	user	action	changes	stamp
197	root@localhost	DELETE	Physics, University of Thessaloniki, ...	2024-01-19T20:05:30
196	root@localhost	DELETE	Nosileutiki, University of Athens, BSc	2024-01-19T20:05:30
195	root@localhost	DELETE	Mousikis, University of Patras, BSc	2024-01-19T20:05:30
194	root@localhost	DELETE	Mechanical Engineering, University o...	2024-01-19T20:05:30
193	root@localhost	DELETE	Mathematics, University of Thessalo...	2024-01-19T20:05:30
192	root@localhost	DELETE	Law, University of Athens, BSc	2024-01-19T20:05:30
191	root@localhost	DELETE	Iatriki, University of Patras, BSc	2024-01-19T20:05:30
190	root@localhost	DELETE	Fysiki, University of Patras, PhD	2024-01-19T20:05:30
189	root@localhost	DELETE	Filologos, University of Athens, BSc	2024-01-19T20:05:30
188	root@localhost	DELETE	Electrical Engineering, University of ...	2024-01-19T20:05:30
187	root@localhost	DELETE	Economics, University of Patras, MSc	2024-01-19T20:05:30
186	root@localhost	DELETE	Economics, University of Athens, BSc	2024-01-19T20:05:30
185	root@localhost	DELETE	Cryptography, University of Athens, P...	2024-01-19T20:05:30
184	root@localhost	DELETE	Computer Engineering and informati...	2024-01-19T20:05:30
183	root@localhost	DELETE	Computer Engineering and informati...	2024-01-19T20:05:30
182	root@localhost	DELETE	Civil Engineering, University of Thes...	2024-01-19T20:05:30
181	root@localhost	DELETE	Business Administration, University ...	2024-01-19T20:05:30
180	root@localhost	DELETE	Biology, University of Patras, PhD	2024-01-19T20:05:30
179	root@localhost	DELETE	zachC, hcaz, Zachary, Chandler, zac...	2024-01-19T20:03:11
178	root@localhost	DELETE	wilmaB, amliw, Wilma, Bush, wilma...	2024-01-19T20:03:11
177	root@localhost	DELETE	Tsosbo, League, Argyris, Koutalas, k...	2024-01-19T20:03:11
176	root@localhost	DELETE	treyE, yert, Trey, Estes, trey@users.db	2024-01-19T20:03:11
175	root@localhost	DELETE	Trakis, trakis190, Dimitris, Giannako...	2024-01-19T20:03:11
174	root@localhost	DELETE	Thanos, tzeleOE, Thanasis, Tzampat...	2024-01-19T20:03:11
173	root@localhost	DELETE	t0p, akyro, Andreas, Tatsopoulos, no...	2024-01-19T20:03:11
172	root@localhost	DELETE	spiros, 1234, Spiros, Papageorgiou, ...	2024-01-19T20:03:11
171	root@localhost	DELETE	Ririto s, Aris1111, Aristeidis, Relos, A...	2024-01-19T20:03:11
170	root@localhost	DELETE	rayS, yar, Ray, Smith, ray@users.db	2024-01-19T20:03:11
169	root@localhost	DELETE	random2, random22, Antonis, Petro...	2024-01-19T20:03:11
168	root@localhost	DELETE	random1, random12, Panagiotis, Ala...	2024-01-19T20:03:11
167	root@localhost	DELETE	PROEDROS, marevaAGAPIMOU, Kyr...	2024-01-19T20:03:11
166	root@localhost	DELETE	Patro4x4, 2003!!2003, Patroklos, Me...	2024-01-19T20:03:11
165	root@localhost	DELETE	Partakias, Dhmosio, Nikos, Androul...	2024-01-19T20:03:11
164	root@localhost	DELETE	Nikolis54, Nick2003, Nikolaos, Andri...	2024-01-19T20:03:11
163	root@localhost	DELETE	megalos, alphaskasi, Thanos, Mikro...	2024-01-19T20:03:11
162	root@localhost	DELETE	markB, kram, Mark, Burgess, mark@...	2024-01-19T20:03:11
161	root@localhost	DELETE	Marika365, Ririka20!, Maria, Karagia...	2024-01-19T20:03:11
160	root@localhost	DELETE	LokoLuke, BamBamPewPew, Louka...	2024-01-19T20:03:11
159	root@localhost	DELETE	Lenio666, elenid1m1triou, Eleni, Di...	2024-01-19T20:03:11
158	root@localhost	DELETE	kostas791, hallaw, Kostas, Papaspy...	2024-01-19T20:03:11
157	root@localhost	DELETE	kellyC, yllek, Kelly, Clayton, kelly@us...	2024-01-19T20:03:11
156	root@localhost	DELETE	iMaxinq, Maxwell420, Maximos, Frou...	2024-01-19T20:03:11
155	root@localhost	DELETE	GiovanniGeorgio, GeoGeoGiGi, Gian...	2024-01-19T20:03:11
154	root@localhost	DELETE	francW, cnarf, Franciszek, Weaver, fr...	2024-01-19T20:03:11
153	root@localhost	DELETE	ellieF, eille, Ellie, Fuentes, ellie@us...	2024-01-19T20:03:11
152	root@localhost	DELETE	cordeliaW, ailrdroc, Cordelia, Watts, ...	2024-01-19T20:03:11
151	root@localhost	DELETE	claireM, erailc, Claire, Mcgowan, clair...	2024-01-19T20:03:11
150	root@localhost	DELETE	ChrisP, chrischris, Xristos, Papaniko...	2024-01-19T20:03:11
149	root@localhost	DELETE	BROEDROS, mercedesBENZ, Aleks...	2024-01-19T20:03:11
148	root@localhost	DELETE	bigMAN, gunned, Stefanos, Xios, pst...	2024-01-19T20:03:11
147	root@localhost	DELETE	BessyLoL, Kotsiri123Kotsiri, Natalia,...	2024-01-19T20:03:11
146	root@localhost	DELETE	anastasiaC, aisatsana, Anastasia, C...	2024-01-19T20:03:11
145	root@localhost	DELETE	Agaphmemnonas21, XrysBrys123, A...	2024-01-19T20:03:11
144	root@localhost	DELETE	Achilleas12, ach1ll789!, Achilleas, P...	2024-01-19T20:03:11
143	root@localhost	UPDATE	New: 6, 2024-04-23, 1200, Junior So...	2024-01-19T19:47:56
142	root@localhost	UPDATE	Old: 6, 2024-04-23, 1200, Junior Soft...	2024-01-19T19:47:56

ΣΥΝΟΛΙΚΟΣ ΚΩΔΙΚΑΣ

```
########################################    CREATE    ########################################
#
DROP DATABASE IF EXISTS Project1;

CREATE DATABASE IF NOT EXISTS Project1;

USE Project1;

CREATE TABLE user
(
```

```sql
  username VARCHAR(30) NOT NULL,
  password VARCHAR(20) NOT NULL,
  name VARCHAR(25) NOT NULL,
  lastname VARCHAR(35) NOT NULL,
  reg_date DATETIME DEFAULT CURRENT_TIMESTAMP NOT NULL,
  email VARCHAR(30) NOT NULL,
  CONSTRAINT user_pk PRIMARY KEY (username)
);

CREATE TABLE admins
(
    username VARCHAR(30) NOT NULL,
    start_date DATETIME DEFAULT CURRENT_TIMESTAMP NOT NULL,
    end_date DATE,
    CONSTRAINT admin_pk PRIMARY KEY (username),
    CONSTRAINT admin_user_fk FOREIGN KEY (username) REFERENCES user(username)
    ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE log
(
    id INT AUTO_INCREMENT,
    user VARCHAR(30) NOT NULL,
    action VARCHAR(10) NOT NULL,
    changes VARCHAR(255) NOT NULL,
    stamp DATETIME NOT NULL,
    CONSTRAINT log_pk PRIMARY KEY (id)
);

CREATE TABLE etairia
(
    AFM CHAR(9) NOT NULL,
    DOY VARCHAR(30) NOT NULL,
    name VARCHAR(35) NOT NULL,
    tel VARCHAR(10) NOT NULL,
    street VARCHAR(15) NOT NULL,
    num INT(11) NOT NULL,
    city VARCHAR(45) NOT NULL,
    country VARCHAR(15) NOT NULL,
    CONSTRAINT etairia_pk PRIMARY KEY (AFM)
);

CREATE TABLE evaluator
(
    username VARCHAR(30) NOT NULL,
    exp_years TINYINT(4) NOT NULL DEFAULT '0',
    firm CHAR(9) NOT NULL,
    CONSTRAINT  eval_pk PRIMARY KEY (username),
    CONSTRAINT  eval_user_fk FOREIGN KEY (username) REFERENCES user(username)
    ON DELETE CASCADE ON UPDATE CASCADE,
```

```sql
    CONSTRAINT  eval_firm_fk FOREIGN KEY (firm) REFERENCES etairia(AFM)
    ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE employee
(
    username VARCHAR(30) NOT NULL,
    bio TEXT,
    sistatikes VARCHAR(35) DEFAULT 'unknown',
    certificates VARCHAR(35) DEFAULT 'unknown',
    CONSTRAINT  employee_pk PRIMARY KEY (username),
    CONSTRAINT  employee_user_fk FOREIGN KEY (username) REFERENCES user(username)
    ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE languages
(
    candid VARCHAR(30) NOT NULL,
    lang SET('EN','FR','SP','GE','CH','GR') NOT NULL,
    CONSTRAINT lang_pk PRIMARY KEY (candid,lang),
    CONSTRAINT lang_fk FOREIGN KEY (candid) REFERENCES employee(username)
    ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE project
(
    candid VARCHAR(30) NOT NULL,
    num TINYINT(4) NOT NULL,
    descr TEXT NOT NULL,
    url VARCHAR(60) DEFAULT 'unknown',
    CONSTRAINT project_pk PRIMARY KEY (candid,num),
    CONSTRAINT project_fk FOREIGN KEY (candid) REFERENCES employee(username)
    ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE job
(
    id INT(11) NOT NULL AUTO_INCREMENT,
    start_date DATE NOT NULL,
    salary FLOAT NOT NULL,
    position VARCHAR(60) NOT NULL,
    edra VARCHAR(60) NOT NULL,
    evaluator VARCHAR(30) NOT NULL,
    grader1 VARCHAR(30),
    grader2 VARCHAR(30),
    announce_date DATETIME DEFAULT CURRENT_TIMESTAMP NOT NULL,
    submission_date DATE AS (DATE_SUB(start_date,INTERVAL 15 DAY)) NOT NULL,
    cancellation_deadline DATE AS (DATE_SUB(start_date,INTERVAL 10 DAY)) NOT NULL,
    CONSTRAINT job_pk PRIMARY KEY (id),
    CONSTRAINT job_eval_fk FOREIGN KEY (evaluator) REFERENCES evaluator(username)
```

```sql
    ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT job_grader1_fk FOREIGN KEY (grader1) REFERENCES evaluator(username)
    ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT job_grader2_fk FOREIGN KEY (grader2) REFERENCES evaluator(username)
    ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE subject
(
    title VARCHAR(36) NOT NULL,
    descr TINYTEXT,
    belongs_to VARCHAR(36) DEFAULT 'none',
    CONSTRAINT subject_pk PRIMARY KEY (title),
    CONSTRAINT req_self_fk FOREIGN KEY (belongs_to) REFERENCES subject(title)
    ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE requires
(
    job_id INT(11) NOT NULL,
    subject_title VARCHAR(36) NOT NULL,
    CONSTRAINT req_pk PRIMARY KEY(job_id,subject_title),
    CONSTRAINT req_job_fk FOREIGN KEY (job_id) REFERENCES job(id)
    ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT req_subj_fk FOREIGN KEY (subject_title) REFERENCES subject(title)
    ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE applies
(
    cand_usrname VARCHAR(30) NOT NULL,
    job_id INT(11) NOT NULL,
    appl_state ENUM('Active','Cancelled') DEFAULT 'Active' NOT NULL,
    appl_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    grade1 TINYINT(4),
    grade2 TINYINT(4),
    CONSTRAINT chk_grade1 CHECK (grade1 between 1 and 20),
    CONSTRAINT chk_grade2 CHECK (grade2 between 1 and 20),
    CONSTRAINT appl_pk PRIMARY KEY (cand_usrname,job_id),
    CONSTRAINT appl_employee_fk FOREIGN KEY (cand_usrname) REFERENCES employee(username)
    ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT appl_job_fk FOREIGN KEY (job_id) REFERENCES job(id)
    ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE application_history
(
    grader1 VARCHAR(30) NOT NULL,
    grader2 VARCHAR(30) NOT NULL,
    cand_usrname VARCHAR(30) NOT NULL,
```

```
    job_id INT(11) NOT NULL,
    appl_state VARCHAR(20) DEFAULT 'Completed' NOT NULL,
    final_grade FLOAT NOT NULL,
    CONSTRAINT appl_history_pk PRIMARY KEY (cand_usrname,job_id),
    CONSTRAINT chk_grade CHECK (final_grade between 0 and 20)
);

CREATE TABLE degree
(
    titlos VARCHAR(150) NOT NULL,
    idryma VARCHAR(150) NOT NULL,
    bathmida ENUM('BSc','MSc','PhD') NOT NULL,
    CONSTRAINT deg_pk PRIMARY KEY (titlos,idryma)
);

CREATE TABLE has_degree
(
    degr_title VARCHAR(150) NOT NULL,
    degr_idryma VARCHAR(140) NOT NULL,
    cand_usrname VARCHAR(30) NOT NULL,
    etos YEAR(4) NOT NULL,
    grade FLOAT NOT NULL,
    CONSTRAINT has_deg_pk PRIMARY KEY (degr_title,degr_idryma,cand_usrname),
    CONSTRAINT has_deg_deg_fk FOREIGN KEY (degr_title,degr_idryma) REFERENCES
degree(titlos,idryma)
    ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT has_deg_employee_fk FOREIGN KEY (cand_usrname) REFERENCES employee(username)
    ON DELETE CASCADE ON UPDATE CASCADE
);

########################################    TRIGGERS    ######################################
###
DROP TRIGGER IF EXISTS degrees_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS degrees_trig  BEFORE INSERT ON has_degree
FOR EACH ROW
BEGIN

    IF NOT EXISTS (SELECT 1 FROM degree WHERE titlos = NEW.degr_title AND idryma =
NEW.degr_idryma) THEN

        INSERT INTO degree(titlos,idryma,bathmida)
        VALUES(NEW.degr_title,NEW.degr_idryma,'BSc');
    END IF;

END$

DELIMITER ;
```

```sql
DROP TRIGGER IF EXISTS projectNumAutoIncrement;


DELIMITER $$
CREATE TRIGGER IF NOT EXISTS projectNumAutoIncrement BEFORE INSERT ON project
FOR EACH ROW
BEGIN
    DECLARE maxNum TINYINT(4);
    SELECT MAX(num) FROM project WHERE candid = NEW.candid INTO maxNum;
    SET NEW.num = IF(ISNULL(maxNum), 1, maxNum + 1);
END$$

DELIMITER ;

#################     JOB INSERT       #########################
DROP TRIGGER IF EXISTS log_job_in_trig;
DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_job_in_trig  AFTER INSERT ON job
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT(NEW.id,', ',
                         NEW.start_date,', ',
                         NEW.salary,', ',
                         NEW.position,', ',
                         NEW.edra,', ',
                         NEW.evaluator);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'INSERT',changes,CURRENT_TIMESTAMP);
END$

DELIMITER ;

#################     JOB UPDATE       #########################
DROP TRIGGER IF EXISTS log_job_up_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_job_up_trig  AFTER UPDATE ON job
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT('Old: ',
                         OLD.id,', ',
                         OLD.start_date,', ',
                         OLD.salary,', ',
                         OLD.position,', ',
                         OLD.edra,', ',
                         OLD.evaluator);
```

```
    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'UPDATE',changes,CURRENT_TIMESTAMP);

    SET changes = CONCAT('New: ',
                         NEW.id,', ',
                         NEW.start_date,', ',
                         NEW.salary,', ',
                         NEW.position,', ',
                         NEW.edra,', ',
                         NEW.evaluator);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'UPDATE',changes,CURRENT_TIMESTAMP);
END$

DELIMITER ;

#################     JOB DELETE       ##########################
DROP TRIGGER IF EXISTS log_job_del_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_job_del_trig  AFTER DELETE ON job
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT(OLD.id,', ',
                         OLD.start_date,', ',
                         OLD.salary,', ',
                         OLD.position,', ',
                         OLD.edra,', ',
                         OLD.evaluator);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'DELETE',changes,CURRENT_TIMESTAMP);
END$

DELIMITER ;
#################     USER INSERT       ##########################
DROP TRIGGER IF EXISTS log_user_in_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_user_in_trig  AFTER INSERT ON user
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT(NEW.username,', ',
                         NEW.password,', ',
                         NEW.name,', ',
                         NEW.lastname,', ',
                         NEW.email);
```

```
    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'INSERT',changes,CURRENT_TIMESTAMP);
END$

DELIMITER ;

#################     USER UPDATE       ##########################
DROP TRIGGER IF EXISTS log_user_up_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_user_up_trig  AFTER UPDATE ON user
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT('Old: ',
                         OLD.username,', ',
                         OLD.password,', ',
                         OLD.name,', ',
                         OLD.lastname,', ',
                         OLD.email);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'UPDATE',changes,CURRENT_TIMESTAMP);

    SET changes = CONCAT('New: ',
                         NEW.username,', ',
                         NEW.password,', ',
                         NEW.name,', ',
                         NEW.lastname,', ',
                         NEW.email);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'UPDATE',changes,CURRENT_TIMESTAMP);
END$

DELIMITER ;

#################     USER DELETE       ##########################
DROP TRIGGER IF EXISTS log_user_del_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_user_del_trig  AFTER DELETE ON user
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT(OLD.username,', ',
                         OLD.password,', ',
                         OLD.name,', ',
                         OLD.lastname,', ',
```

```
                        OLD.email);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'DELETE',changes,CURRENT_TIMESTAMP);
END$

DELIMITER ;

#################     DEGREE INSERT       ##########################
DROP TRIGGER IF EXISTS log_degree_in_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_degree_in_trig  AFTER INSERT ON degree
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT(NEW.titlos,', ',
                        NEW.idryma,', ',
                        NEW.bathmida);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'INSERT',changes,CURRENT_TIMESTAMP);
END$

DELIMITER ;

#################     DEGREE UPDATE       ##########################
DROP TRIGGER IF EXISTS log_degree_up_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_degree_up_trig  AFTER UPDATE ON degree
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT('Old: ',
                        OLD.titlos,', ',
                        OLD.idryma,', ',
                        OLD.bathmida);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'UPDATE',changes,CURRENT_TIMESTAMP);

    SET changes = CONCAT('New: ',
                        NEW.titlos,', ',
                        NEW.idryma,', ',
                        NEW.bathmida);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'UPDATE',changes,CURRENT_TIMESTAMP);
END$
```

```mysql
DELIMITER ;

#################     DEGREE DELETE       ##########################
DROP TRIGGER IF EXISTS log_degree_del_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS log_degree_del_trig  AFTER DELETE ON degree
FOR EACH ROW
BEGIN
    DECLARE changes VARCHAR(255);
    SET changes = CONCAT(OLD.titlos,', ',
                         OLD.idryma,', ',
                         OLD.bathmida);

    INSERT INTO log(user,action,changes,stamp)
    VALUES(CURRENT_USER(),'DELETE',changes,CURRENT_TIMESTAMP);
END$

DELIMITER ;


DROP TRIGGER IF EXISTS applies_in_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS applies_in_trig  BEFORE INSERT ON applies
FOR EACH ROW
BEGIN
    DECLARE submission_date DATE;
    DECLARE active_applications TINYINT(1);

    ########    sub_date    ########
    SELECT job.submission_date INTO submission_date
    FROM job
    WHERE job.id = NEW.job_id;

    IF CURRENT_DATE > submission_date THEN # CURRENT_DATE is the date an applicant attempts
to apply
        SIGNAL SQLSTATE VALUE '45000'
        SET MESSAGE_TEXT = 'Application date too close to job start date.';
    END IF;

    ########    cand active applications    ########
    SELECT count(appl_state) INTO active_applications
    FROM applies
    WHERE appl_state = 'Active' AND cand_usrname = NEW.cand_usrname;

    IF active_applications = 3 THEN
        SIGNAL SQLSTATE VALUE '45000'
        SET MESSAGE_TEXT = 'Applicant already has three active applications.';
```

```
    END IF;
END$

DELIMITER ;

DROP TRIGGER IF EXISTS applies_cancel_trig;

DELIMITER $
CREATE TRIGGER IF NOT EXISTS applies_cancel_trig  BEFORE UPDATE ON applies
FOR EACH ROW
BEGIN

    DECLARE cancellation_deadline DATE;
    DECLARE active_applications TINYINT(1);

    IF NEW.appl_state = 'Cancelled' THEN

        ########    cancel_date    ########
        SELECT job.cancellation_deadline INTO cancellation_deadline
        FROM job
        WHERE job.id = NEW.job_id;

        IF CURRENT_DATE > cancellation_deadline THEN # CURRENT_DATE is the date an applicant
attempts to cancel
            SIGNAL SQLSTATE VALUE '45000'
            SET MESSAGE_TEXT = 'Cancelation date too close to job start date.';
        END IF;
    END IF;

    IF NEW.appl_state = 'Active' AND OLD.appl_state = 'Cancelled' THEN

        ########    cand active applications    ########
        SELECT count(appl_state) INTO active_applications
        FROM applies
        WHERE appl_state = 'Active' AND cand_usrname = NEW.cand_usrname;

        IF active_applications = 3 THEN
            SIGNAL SQLSTATE VALUE '45000'
            SET MESSAGE_TEXT = 'Applicant already has three active applications.';
        END IF;
    END IF;
END$

DELIMITER ;
########################################    PROCEDURES    ##################################
#####
DROP PROCEDURE IF EXISTS calcGrade;
DELIMITER $$
CREATE PROCEDURE calcGrade(IN eval_username_par VARCHAR(30), IN empl_username_par
VARCHAR(30), IN job_parameter  INT, OUT grade_parameter INT)
```

```
BEGIN
    DECLARE known_languages VARCHAR(30);
    DECLARE project_count TINYINT;
    DECLARE bsc_count TINYINT(4);
    DECLARE msc_count TINYINT(4);
    DECLARE phd_count TINYINT(4);

    SELECT MAX(num) INTO project_count
    FROM job INNER JOIN applies ON job.id=applies.job_id INNER JOIN employee ON
applies.cand_usrname=employee.username INNER JOIN
    project ON employee.username=project.candid
    WHERE job_parameter=applies.job_id AND (eval_username_par=job.grader1 OR
eval_username_par=job.grader2) AND empl_username_par=applies.cand_usrname;

    SELECT lang INTO known_languages
    FROM job INNER JOIN applies ON job.id=applies.job_id INNER JOIN employee ON
applies.cand_usrname=employee.username INNER JOIN
    languages ON employee.username=languages.candid
    WHERE job_parameter=applies.job_id AND (eval_username_par=job.grader1 OR
eval_username_par=job.grader2) AND empl_username_par=applies.cand_usrname;

    SELECT COUNT(bathmida) INTO bsc_count
    FROM job INNER JOIN applies ON job.id=applies.job_id INNER JOIN employee ON
applies.cand_usrname=employee.username
    INNER JOIN has_degree ON employee.username=has_degree.cand_usrname INNER JOIN degree ON
has_degree.degr_title=degree.titlos AND has_degree.degr_idryma=degree.idryma
    WHERE job_parameter=applies.job_id AND (eval_username_par=job.grader1 OR
eval_username_par=job.grader2) AND
    empl_username_par=applies.cand_usrname AND degree.bathmida='BSc'
    GROUP BY bathmida;

    SELECT COUNT(bathmida) INTO msc_count
    FROM job INNER JOIN applies ON job.id=applies.job_id INNER JOIN employee ON
applies.cand_usrname=employee.username
    INNER JOIN has_degree ON employee.username=has_degree.cand_usrname INNER JOIN degree ON
has_degree.degr_title=degree.titlos AND has_degree.degr_idryma=degree.idryma
    WHERE job_parameter=applies.job_id AND (eval_username_par=job.grader1 OR
eval_username_par=job.grader2) AND
    empl_username_par=applies.cand_usrname AND degree.bathmida='MSc'
    GROUP BY bathmida;

    SELECT COUNT(bathmida) INTO phd_count
    FROM job INNER JOIN applies ON job.id=applies.job_id INNER JOIN employee ON
applies.cand_usrname=employee.username
    INNER JOIN has_degree ON employee.username=has_degree.cand_usrname INNER JOIN degree ON
has_degree.degr_title=degree.titlos AND has_degree.degr_idryma=degree.idryma
    WHERE job_parameter=applies.job_id AND (eval_username_par=job.grader1 OR
eval_username_par=job.grader2) AND
    empl_username_par=applies.cand_usrname AND degree.bathmida='PhD'
    GROUP BY bathmida;
```

```
    IF (INSTR(known_languages,',')>0) THEN
        SET
grade_parameter=IFNULL(project_count,0)+IFNULL(bsc_count,0)+2*IFNULL(msc_count,0)+3*IFNULL(ph
d_count,0)+1;
    ELSE
        SET
grade_parameter=IFNULL(project_count,0)+IFNULL(bsc_count,0)+2*IFNULL(msc_count,0)+3*IFNULL(ph
d_count,0);
    END IF;
END $$
DELIMITER ;

##########################################################################################
#######################################################################

DROP PROCEDURE IF EXISTS grading;
DELIMITER $$
CREATE PROCEDURE grading(IN eval_username VARCHAR(30), IN empl_username VARCHAR(30), IN
job_param  INT, OUT grade_param INT)
BEGIN
    DECLARE vathmologitis1 VARCHAR(30);
    DECLARE vathmologitis2 VARCHAR(30);
    DECLARE vathmos1 TINYINT(4);
    DECLARE vathmos2 TINYINT(4);

    SELECT grader1,grader2,grade1,grade2 INTO vathmologitis1,vathmologitis2,vathmos1,vathmos2
    FROM job INNER JOIN applies ON job.id=applies.job_id
    WHERE  job_param=applies.job_id AND (eval_username=job.grader1 OR
eval_username=job.grader2) AND empl_username=applies.cand_usrname;

    IF (vathmologitis1=eval_username) THEN
        IF (vathmos1 IS NOT NULL) THEN
            SET grade_param=vathmos1;
        ELSE
            CALL calcGrade(eval_username,empl_username,job_param,grade_param);
        END IF;

    ELSEIF (vathmologitis2=eval_username) THEN
        IF (vathmos2 IS NOT NULL) THEN
            SET grade_param=vathmos2;
        ELSE
            CALL calcGrade(eval_username,empl_username,job_param,grade_param);
        END IF;

    ELSE
        SET grade_param=0;
    END IF;
END $$
DELIMITER ;
```

```
DROP PROCEDURE IF EXISTS applyProcedure;
DELIMITER $
CREATE PROCEDURE applyProcedure(IN p_username VARCHAR(30), IN p_jobID INT, IN charX
ENUM('i','c','a'))
BEGIN
    DECLARE temp_grader1 VARCHAR(30);
    DECLARE temp_grader2 VARCHAR(30);
    DECLARE temp_firm VARCHAR(9);
    DECLARE temp_candUsrname VARCHAR(30);
    DECLARE temp_jobID INT;
    DECLARE temp_state VARCHAR(12);


    CASE charX
        WHEN 'i' THEN
            SELECT grader1,grader2 INTO temp_grader1,temp_grader2
            from employee INNER JOIN applies on employee.username = applies.cand_usrname
            INNER JOIN job ON applies.job_id = job.id
            WHERE p_username = employee.username AND p_jobID = job.id;

            IF temp_grader1 IS NULL THEN
                IF temp_grader2 IS NULL THEN
                    UPDATE job SET grader1 = 'GiovanniGeorgio' ,grader2 = 'Lenio666' WHERE
id=p_jobID;
                ELSE
                    SELECT firm into temp_firm
                    FROM evaluator
                    WHERE evaluator.username = temp_grader2;

                    SELECT username INTO temp_grader1
                    FROM evaluator
                    WHERE evaluator.firm = temp_firm AND evaluator.username != temp_grader2
                    LIMIT 1;

                    UPDATE job SET grader1 = temp_grader1 WHERE id=p_jobID;
                END IF;

            ELSE
                IF temp_grader2 IS NULL THEN
                    SELECT firm into temp_firm
                    FROM evaluator
                    WHERE evaluator.username = temp_grader1;

                    SELECT username INTO temp_grader2
                    FROM evaluator
                    WHERE evaluator.firm = temp_firm AND evaluator.username != temp_grader1
                    LIMIT 1;
```

```
                    UPDATE job SET grader2 = temp_grader2 WHERE id=p_jobID;
                END IF;
            END IF;

            INSERT INTO applies(cand_usrname,job_id,appl_state)
            VALUES (p_username,p_jobID,'Active');

        WHEN 'c' THEN
            SELECT cand_usrname,job_id INTO temp_candUsrname,temp_jobID
            FROM applies
            WHERE cand_usrname = p_username AND applies.job_id = p_jobID AND appl_state !=
'Cancelled';

            IF temp_jobID IS NULL THEN
                SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Application doesnt exist or has been cancelled';
            END IF;

            UPDATE applies SET appl_state = 'Cancelled' WHERE cand_usrname = temp_candUsrname
AND job_id = temp_jobID;


        WHEN 'a' THEN
            SELECT cand_usrname,job_id INTO temp_candUsrname,temp_jobID
            FROM applies
            WHERE cand_usrname = p_username AND applies.job_id = p_jobID AND appl_state =
'Cancelled';

            IF temp_candUsrname IS NOT NULL THEN
                UPDATE applies SET appl_state = 'Active' WHERE cand_usrname =
temp_candUsrname AND job_id = temp_jobID;
                SELECT 'Successfully updated application' AS message;
            ELSE
                SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'No cancelled application exists';

            END IF;

    END CASE;
END $
DELIMITER ;


DROP PROCEDURE IF EXISTS fillGrades;
DELIMITER $$
CREATE PROCEDURE fillGrades(IN input_job INT)
BEGIN

    DECLARE finished INT DEFAULT 0;
    DECLARE applicant VARCHAR(30);
```

```
    DECLARE evaluator1 VARCHAR(30);
    DECLARE evaluator2 VARCHAR(30);
    DECLARE calculated_grade1 TINYINT(4);
    DECLARE calculated_grade2 TINYINT(4);

    DECLARE cursor_applicants CURSOR FOR
    SELECT cand_usrname,grader1,grader2
    FROM applies INNER JOIN job ON job.id=applies.job_id
    WHERE applies.job_id=input_job AND applies.appl_state = 'Active' AND (applies.grade1 IS
NULL OR applies.grade2 IS NULL);

    DECLARE CONTINUE HANDLER FOR NOT FOUND
    SET finished=1;

    OPEN cursor_applicants;
    REPEAT

        FETCH cursor_applicants INTO applicant,evaluator1,evaluator2;
        CALL grading(evaluator1,applicant,input_job,calculated_grade1);
        CALL grading(evaluator2,applicant,input_job,calculated_grade2);

        UPDATE applies
        SET grade1=calculated_grade1, grade2=calculated_grade2
        WHERE cand_usrname=applicant AND job_id=input_job;

    UNTIL finished=1
    END REPEAT;
    CLOSE cursor_applicants;

END $$
DELIMITER ;

##########################################################################################
#######################################################################

DROP PROCEDURE IF EXISTS findEmployee;
DELIMITER $$
CREATE PROCEDURE findEmployee(IN job_ID INT)
BEGIN

    CALL fillGrades(job_ID);

    SELECT cand_usrname,(grade1+grade2)/2 AS grade
    FROM applies
    WHERE applies.job_id = job_ID AND applies.appl_state = 'Active'
    ORDER BY grade DESC,appl_date ASC
    LIMIT 1;

    INSERT INTO application_history (cand_usrname,grader1,grader2,job_id,final_grade)
```

```
        SELECT
applies.cand_usrname,job.grader1,job.grader2,applies.job_id,(applies.grade1+applies.grade2)/2
        FROM applies INNER JOIN job ON job.id=applies.job_id
        WHERE applies.job_id=job_ID AND applies.appl_state = 'Active';

    INSERT INTO application_history (cand_usrname,grader1,grader2,job_id,final_grade)
        SELECT applies.cand_usrname,job.grader1,job.grader2,applies.job_id,0
        FROM applies INNER JOIN job ON job.id=applies.job_id
        WHERE applies.job_id=job_ID AND applies.appl_state = 'Cancelled';

    DELETE FROM applies
    WHERE  applies.job_id=job_ID;

END $$
DELIMITER ;


DROP PROCEDURE IF EXISTS gradeRange;

DELIMITER $

CREATE PROCEDURE gradeRange(IN grade1 FLOAT, IN grade2 FLOAT)
BEGIN
    SELECT cand_usrname AS applicant, job_id AS job
    FROM application_history
    WHERE final_grade BETWEEN grade1 AND grade2;
END$
DELIMITER ;

ALTER TABLE application_history
ADD INDEX grade_idx (final_grade);

##########################################################################################
#######################################################################

DROP PROCEDURE IF EXISTS evaluatedBy;

DELIMITER $

CREATE PROCEDURE evaluatedBy(IN eval_usrname VARCHAR(30))
BEGIN
    SELECT cand_usrname AS applicant, job_id AS job
    FROM application_history
    WHERE grader1 = eval_usrname OR grader2 = eval_usrname;
END$
DELIMITER ;

ALTER TABLE application_history
ADD INDEX grader_idx (grader1,grader2);
```

```sql
-- EXPLAIN SELECT cand_usrname AS applicant, job_id AS job
-- FROM application_history
-- WHERE final_grade BETWEEN 1 AND 20;

-- EXPLAIN SELECT cand_usrname AS applicant, job_id AS job
-- FROM application_history
-- WHERE grader1 = 'seceval70000' OR grader2 = 'seceval70000';


DROP PROCEDURE IF EXISTS clearTable;

DELIMITER $$
CREATE PROCEDURE IF NOT EXISTS clearTable(IN tableName VARCHAR(20))
BEGIN
    SET @x = CONCAT('DELETE FROM ',tableName); # Making querie string

    PREPARE stmnt FROM @x;      # Making the string an actual sql querie
    EXECUTE stmnt;              # Executing the querie we made
    DEALLOCATE PREPARE stmnt;   # Deallocating the memory from the statement we made and
executed

END$$
DELIMITER ;


DROP PROCEDURE IF EXISTS clearDB;

DELIMITER $$
CREATE PROCEDURE IF NOT EXISTS clearDB()
BEGIN
    CALL clearTable('user');
    CALL clearTable('subject');
    CALL clearTable('etairia');
    CALL clearTable('degree');
    CALL clearTable('application_history');
END$$
DELIMITER ;
########################################    INSERT    ########################################
#
INSERT INTO user(username,password,name,lastname,email)
VALUES('SpirosP','1234','Spiros','Papageorgiou','spiros@users.db'),
      ('markB','kram','Mark','Burgess','mark@users.db'),
      ('treyE','yert','Trey','Estes','trey@users.db'),
      ('cordeliaW','ailrdroc','Cordelia','Watts','cordel@users.db'),
      ('wilmaB','amliw','Wilma','Bush','wilma@users.db'),
      ('anastasiaC','aisatsana','Anastasia','Chan','ana@users.db'),
      ('kellyC','yllek','Kelly','Clayton','kelly@users.db'),
```

```sql
      ('francW','cnarf','Franciszek','Weaver','franc@users.db'),
      ('ellieF','eille','Ellie','Fuentes','ellie@users.db'),
      ('claireM','erailc','Claire','Mcgowan','claire@users.db'),
      ('rayS','yar','Ray','Smith','ray@users.db'),
      ('zachC','hcaz','Zachary','Chandler','zach@users.db')
;

INSERT INTO admins(username)
VALUES('claireM'),
      ('ellieF')
;


INSERT INTO employee(username,bio,sistatikes,certificates)
VALUES('SpirosP',NULL,'sistat_Spiros.pdf','cert_Spiros.pdf'),
      ('markB',NULL,'sistat_Mark.pdf','cert_Mark.pdf'),
      ('treyE',NULL,'sistat_Trey.pdf','cert_Trey.pdf'),
      ('cordeliaW',NULL,'sistat_Cord.pdf','cert_Cord.pdf'),
      ('wilmaB',NULL,'sistat_Wil.pdf','cert_Wil.pdf'),
      ('anastasiaC',NULL,'sistat_Ana.pdf','cert_Ana.pdf')
;


INSERT INTO etairia(AFM,DOY,name,tel,street,num,city,country)
VALUES('a','Patron','et1','1111111111','st1','1','Patra','Greece'),
      ('b','Athinon','et2','2222222222','st2','2','Athina','Greece'),
      ('c','Thesalonikis','et3','3333333333','st3','3','Thesaloniki','Greece')
;


INSERT INTO evaluator(username,exp_years,firm)
VALUES('kellyC',0,'a'),
      ('francW',1,'b'),
      ('ellieF',2,'b'),
      ('claireM',3,'a'),
      ('rayS',4,'c'),
      ('zachC',5,'c')
;


INSERT INTO degree(titlos,idryma,bathmida)
VALUES('Computer Engineering and informatics','University of Patras','MSc'),
      ('Electrical Engineering','University of Patras','BSc'),
      ('Law','University of Athens','BSc'),
      ('Mechanical Engineering','University of Patras','PhD'),
      ('Biology','University of Patras','PhD'),
      ('Economics','University of Athens','BSc')
;


INSERT INTO has_degree(degr_title,degr_idryma,cand_usrname,etos,grade)
```

```sql
VALUES('Computer Engineering and informatics','University of Patras','SpirosP',2022,8.5),
      ('Law','University of Athens','markB',2017,9),
      ('Economics','University of Athens','treyE',2020,7.4),
      ('Mechanical Engineering','University of Patras','cordeliaW',2019,7.6),
      ('Electrical Engineering','University of Patras','anastasiaC',2015,7),
      ('Biology','University of Patras','wilmaB',2021,6.7)
;



INSERT INTO job(start_date,salary,position,edra,evaluator)
VALUES('2024-03-01',3500.0,'Manager','Athens','kellyC'),
      ('2024-02-23',2200.0,'Researcher','Athens','rayS'),
      ('2024-03-15',1400.0,'Intermediate Software Engineer','Thesaloniki','ellieF'),
      ('2024-05-14',1500.0,'Intermediate Software Engineer','Thesaloniki','francW'),
      ('2024-02-18',1400.0,'Junior Software Engineer','Patra','claireM'),
      ('2024-04-23',1200.0,'Junior Software Engineer','Athens','zachC'),
      ('2024-04-01',2700.0,'Assistant Attorney','Athens','zachC'),
      ('2024-06-01',1000.0,'Internship','Patra','zachC')
;

INSERT INTO applies(cand_usrname,job_id,appl_state)
VALUES('SpirosP',1,'Active'),
      ('SpirosP',3,'Cancelled'),
      ('SpirosP',4,'Active'),
      ('markB',7,'Cancelled'),
      ('cordeliaW',8,'Active'),
      ('wilmaB',5,'Cancelled'),
      ('wilmaB',2,'Active')
;


INSERT INTO languages(candid,lang)
VALUES('treyE','GR,EN,FR'),
      ('cordeliaW','GR,EN'),
      ('wilmaB','GR,EN,SP,GE'),
      ('anastasiaC','GR,EN,CH')
;


INSERT INTO project(candid,descr,url)
VALUES('SpirosP',"Created a database for hotel
reservations.","www.github.com/SpirosP/code/"),
      ('SpirosP',"A ML application to predict the weather.","www.github.com/SpirosP/code/"),
      ('markB',"Participated in trial for human rights
violations","www.github.com/markB/code/"),
      ('treyE',"Mathematical predictions for stock prices.","www.github.com/treyE/code/"),
      ('cordeliaW',"Design of a small turbine engine in colaboration with
ESA.","www.github.com/cordeliaW/code/"),
      ('treyE',"Statistical analysis of investments for
AlphaBank.","www.github.com/treyE/code/"),
```

```
      ('anastasiaC',"Research on local fauna","www.github.com/anastasiaC/code/")
;


INSERT INTO subject(title,descr,belongs_to)
VALUES('Computer Engineering',"Hardware and Software",NULL),
      ('Database Programming',"Principles of Database design and use.",'Computer
Engineering'),
      ('DML'," Data manipulation languages are computer programming languages used for
inserting,
                 deleting, and updating data in a database.An example is SQL
language",'Database Programming'),

      ('Computer Architecture',"Configuration of computer hardware.",'Computer Engineering'),
      ('HDL',"Hardware description language to design digital logic.",'Computer
Architecture'),

      ('Mechanical Engineering',"Mechanical and aerospace engineering",NULL),
      ('Fluid Mechanics',"the study of fluid behavior at rest and in motion. Fluid mechanics
has a wide range
            of applications in mechanical and chemical engineering, in biological systems,
and in astrophysics.",'Mechanical Engineering'),

      ('Artificial Intelligence',"Efficient algorithms used in multiple
applications",'Computer Engineering'),
      ('Machine  Learning',"A part of artificial intelligence used to teach computers to do
what comes naturally to humans",'Artificial Intelligence'),

      ('Law',"Greek Law",NULL),
      ('International Law',"Set of rules, norms, and standards generally recognized as
binding between states. It establishes
            norms for states across a broad range of domains, including war and diplomacy,
economic relations, and human rights.",'Law')
;


INSERT INTO requires(job_id,subject_title)
VALUES(7,'International Law'),
      (1,'Computer Architecture'),
      (2,'Machine  Learning'),
      (3,'Database Programming'),
      (4,'Database Programming'),
      (5,'HDL'),
      (6,'HDL'),
      (8,'Fluid Mechanics')
;


INSERT INTO user(username,password,name,lastname,email)
VALUES
```

```sql
      ('kostas791','hallaw','Kostas','Papaspyrou','olikostas@gmail.com'),
      ('Thanos','tzeleOE','Thanasis','Tzampatzis','thanasis2003@gmail.com'),
      ('Trakis','trakis190','Dimitris','Giannakopoylos','zaxari@gmail.com'),
      ('t0p','akyro','Andreas','Tatsopoulos','notAndrew@gmail.com'),
      ('Tsosbo','League','Argyris','Koutalas','koutalas@gmail.com'),
      ('bigMAN','gunned','Stefanos','Xios','pstSipsas@gmail.com'),
      ('megalos','alphaskasi','Thanos','Mikroutsikos','mikroutsikos@gmail.com'),
      ('PROEDROS','marevaAGAPIMOU','Kyriakos','Mitsotakis','kommaPasisEllados@gmail.com'),
      ('Partakias','Dhmosio','Nikos','Androulakis','epidomaYparksis@gmail.com'),
      ('BROEDROS','mercedesBENZ','Aleksis','Tsipras','strofi360moires@gmail.com'),
      ('random1','random12','Panagiotis','Alafaouzeos','trifuli@gmail.com'),
      ('random2','random22','Antonis','Petrou','arsak@gmail.com');

INSERT INTO admins(username)
VALUES
      ('kostas791'),
      ('PROEDROS');


INSERT INTO employee(username,bio,sistatikes,certificates)
VALUES
      ('Partakias',NULL,'sistat_Partakias.pdf','cert_Partakias.pdf'),
      ('Thanos',NULL,'sistat_Thanos.pdf','cert_Thanos.pdf'),
      ('BROEDROS',NULL,'sistat_BROEDROS.pdf','cert_BROEDROS.pdf'),
      ('megalos',NULL,'sistat_megalos.pdf','cert_megalos.pdf'),
      ('bigMAN',NULL,'sistat_bigMAN.pdf','cert_Tsosbo.pdf'),
      ('random1',NULL,'sistat_random1.pdf','cert_random1.pdf');


INSERT INTO etairia(AFM,DOY,name,tel,street,num,city,country)
VALUES
      ('d','Metsobou','et4','4444444444','st4','4','Metsobo','Greece'),
      ('e','Artas','et5','5555555555','st5','5','Arta','Greece'),
      ('f','Trikalwn','et6','6666666666','st6','6','Trikala','Greece');


INSERT INTO evaluator(username,exp_years,firm)
VALUES
      ('kostas791',9,'d'),
      ('PROEDROS',9,'f'),
      ('t0p',2,'e'),
      ('Trakis',4,'f'),
      ('TSOSBO',0,'d'),
      ('random2',3,'e');


INSERT INTO degree(titlos,idryma,bathmida)
VALUES
        ('Filologos','University of Athens','BSc'),
        ('Mousikis','University of Patras','BSc'),
        ('Cryptography','University of Athens','PhD'),
```

```sql
        ('Iatriki','University of Patras','BSc'),
        ('Nosileutiki','University of Athens','BSc'),
        ('Fysiki','University of Patras','PhD');


INSERT INTO has_degree(degr_title,degr_idryma,cand_usrname,etos,grade)
VALUES
      ('Filologos','University of Athens','Partakias',2021,6),
      ('Mousikis','University of Patras','Thanos',2018,9),
      ('Cryptography','University of Athens','BROEDROS',2020,10),
      ('Fysiki','University of Patras','random1',2023,7);


INSERT INTO job(start_date,salary,position,edra,evaluator)
VALUES
      ('2024-04-12','2300.0','Mousikos se sxoleio','Patra','kostas791'),
      ('2024-05-11','2100.0','Filologos se sxoleio','Patra','PROEDROS'),
      ('2024-06-09','4100.0','Kathigitis Kryptografias','Athens','t0p'),
      ('2024-07-28','700.0','Mousikos se sxoleio','Thesaloniki','Trakis'),
      ('2024-08-29','1000.0','Analytis keimenwn','Patra','TSOSBO'),
      ('2024-10-12','800.0','Fysikos se sxoleio','Athens','random2'),
      ('2024-11-07','750.0','Fysikos se frontistirio','Patra','t0p'),
      ('2024-08-16','950.0','Cybersecutiry engineer','Thesaloniki','kostas791');

INSERT INTO applies(cand_usrname,job_id,appl_state)
VALUES
      ('Partakias',9,'Active'),
      ('BROEDROS',11,'Active'),
      ('megalos',12,'Cancelled'),
      ('Thanos',10,'Active'),
      ('bigMAN',13,'Active'),
      ('Thanos',14,'Cancelled'),
      ('Partakias',15,'Active'),
      ('BROEDROS',16,'Active'),
      ('megalos',9,'Cancelled');


INSERT INTO languages(candid,lang)
VALUES
      ('Partakias','GR,EN'),
      ('Thanos','GR,FR'),
      ('megalos','GR'),
      ('BROEDROS','GR,EN,GE');


INSERT INTO project(candid,descr,url)
VALUES
      ('Partakias',"Kati me ypologistes","github.com/Partakias/"),
      ('BROEDROS',"Kati me mousiki","mousiki.gr/BROEDROS/"),
      ('Thanos',"Graphic Design kai tetoia","thanossite.gr/thanos/"),
      ('megalos',"Pos na ftiakseis ton teleio kafe","Kilimanjaro.gr/megalos"),
```

```sql
      ('bigMAN',"Advanced pliroforiaka systimata","ceid.gr/bigMAN/"),
      ('Partakias',"Made a pentesting tool","github.com/kostas791/"),
      ('BROEDROS',"Made a C library","github.com/kostas791/");


INSERT INTO subject(title,descr,belongs_to)
VALUES
    ('Music','Na kserei na paizei ena organo',NULL),
    ('Literature','Na kserei na kanei analysh keimenou',NULL),
    ('Cryptography','Na kserei tis arxes kryptografias',NULL),
    ('Fysiki','Na kserei na kanei fysiki',NULL),
    ('Sxetikotitas','Na kserei theoria sxetikotitas','Fysiki'),
    ('AES','Na kserei pos douleuei o AES','Cryptography'),
    ('Dynameis','Na kserei analysh dynamewn se kinoumeno antikeimeno','Fysiki'),
    ('Keimenografos','Na mporei na kanei diorthosi keimenou','Literature');

INSERT INTO requires(job_id,subject_title)
VALUES
      (9,'Music'),
      (10,'Literature'),
      (11,'Cryptography'),
      (12,'Music'),
      (13,'Literature'),
      (14,'Sxetikotitas'),
      (15,'Dynameis'),
      (16,'AES');



INSERT INTO user(username,password,name,lastname,email)
VALUES('iMaxinq','Maxwell420','Maximos','Frountzos','MaxwellFr@gmail.com'),
      ('BessyLoL','Kotsiri123Kotsiri','Natalia','Kotsiri','BessyKots@gmail.com'),
      ('LokoLuke','BamBamPewPew','Loukas','Tyxeropoulos','LoukasTyx@gmail.com'),
      ('GiovanniGeorgio','GeoGeoGiGi','Giannis','Georgiou','GiovanniGeo@gmail.com'),
      ('Nikolis54','Nick2003','Nikolaos','Andrikopoulos','NikolasAndr@gmail.com'),
      ('Marika365','Ririka20!','Maria','Karagianni','MariaKar@gmail.com'),
      ('Riritos','Aris1111','Aristeidis','Relos','AristeidRel@gmail.com'),
      ('ChrisP','chrischris','Xristos','Papanikolaou','ChrisPapan@gmail.com'),
      ('Lenio666','elenid1m1triou','Eleni','Dimitriou','EleniDim@gmail.com'),
      ('Achilleas12','ach1ll789!','Achilleas','Phleidis','AchilPhl@gmail.com'),
      ('Patro4x4','2003!!2003','Patroklos','Menoitiou','PatrMen@gmail.com'),
      ('Agaphmemnonas21','XrysBrys123','Agamemnonas','Atreidis','AgamemnAtr@gmail.com');

INSERT INTO etairia(AFM,DOY,name,tel,street,num,city,country)
VALUES('109350999','Patron','Volksvaggos','2610222420','Kolokotroni','1','Patra','Greece'),
    ('620785420','Athinon','Sutzuki','2610930301','Karolou','75','Athina','Greece'),
      ('991601690','Thesalonikis','AlfaIoylieta','2610220342','Kanakari','334','Thesaloniki',
'Greece');

INSERT INTO evaluator(username,exp_years,firm)
```

```
VALUES('BessyLoL',1,'109350999'),
    ('GiovanniGeorgio',5,'991601690'),
      ('Riritos',2,'109350999'),
    ('Lenio666',1,'991601690'),
      ('Achilleas12',7,'620785420'),
    ('Patro4x4',3,'109350999');

INSERT INTO employee(username,bio,sistatikes,certificates)
VALUES('iMaxinq',"O Maximos Frountzos spoydase sto CEID sthn Patra, doylepse gia 4 xronia
sthn AMD.",'sistat_Max.pdf','cert_Max.pdf'),
    ('LokoLuke',"O Loukas Tyxeropoulos spoydase sto tmhma oikonomikwn epistimwn sthn Athina
opoy kai apofitise me arista",'sistat_Louk.pdf','cert_Louk.pdf'),
      ('Nikolis54',"O Nikolaos Andrikopoulos spoydase sto tmhma Mathimatikwn kai epeita
spoudase sto tmhma fysikhs
       sthn Thessaloniki kai idryse thn MathLab.",'sistat_Nikolas.pdf','cert_Nikolas.pdf'),
    ('Marika365',"H Maria Karagianni spoydase dioikish twn epixeirisewn sthn
Athina.",'sistat_Maria.pdf','cert_Maria.pdf'),
      ('ChrisP',"O Christos Papanikolaou prokeitai gia autodidakto programmatisth me empeiria
3 etwn.",'sistat_Xristos.pdf','cert_Xristos.pdf'),
    ('Agaphmemnonas21',"O Agamemnonas Atreidis spoydase sthn Thessaloniki politikos
mixanikos",'sistat_Agamemn.pdf','cert_Agamemn.pdf');

INSERT INTO languages(candid,lang)
VALUES('iMaxinq','GR,EN,FR,SP,GE'),
    ('LokoLuke','GR'),
      ('Nikolis54','GR,EN,SP,GE'),
    ('Agaphmemnonas21','GR,CH');

INSERT INTO project(candid,descr,url)
VALUES('iMaxinq',"Created a database for a travelling
agency.","www.github.com/iMaxinq/code/"),
    ('Nikolis54',"Did the calculations for a required weather
prediction","www.github.com/Nikolis54/code/"),
      ('Marika365',"Took the necessary actions for increasing the company's
profits","www.github.com/Marika365/code/"),
    ('iMaxinq',"Mathematical predictions for stock prices.","www.github.com/iMaxinq/code/"),
      ('Agaphmemnonas21',"Designed the blueprint for an extension to the company's main
building","www.github.com/Agaphmemnonas21/code/"),
    ('LokoLuke',"Statistical analysis of investments for the
company","www.github.com/LokoLuke/code/"),
      ('ChrisP',"Made the company's website","www.github.com/ChrisP/code/");

INSERT INTO job(start_date,salary,position,edra,evaluator)
VALUES('2024-02-18',3500.0,'Manager','Athens','Lenio666'),
      ('2024-02-20',3100.0,'Mathematician','Patras','Riritos'),
      ('2024-09-15',1300.0,'Software Engineer','Patras','GiovanniGeorgio'),
      ('2024-02-24',1600.0,'Software Engineer','Thessaloniki','Lenio666'),
      ('2024-11-18',1100.0,'Accountant','Patras','BessyLoL'),
      ('2024-04-23',1100.0,'Software Engineer','Thessaloniki','Riritos'),
      ('2024-04-01',2760.0,'Software Engineer','Athens','Achilleas12'),
```

```sql
      ('2024-12-01',2100.0,'Civil Engineer','Patras','Patro4x4');

INSERT INTO subject(title,descr,belongs_to)
VALUES('Statistics',"Collection of data to predict future behaviors",NULL),
      ('Numerical Analysis',"Development of efficient algorithms through numerical
approximation for solving mathematical problems",NULL),
      ('Programming',"Creating instructions for a computer to understand and execute",NULL),
      ('Website development',"Writing code for websites",'Programming'),
      ('Logistics',"Planning, coordination, and implementation of the flow of goods,
services, and information from their origin to consumption",NULL),
      ('AI',"Algorithms that simulate human behavior",NULL),
      ('Root-Finding Algorithms',"A part of Numerical analysis that deals with methods to
find roots of equations",'Numerical Analysis'),
      ('Newtonian Physics research',"Fundamental laws regarding the motion of objects",NULL);

INSERT INTO requires(job_id,subject_title)
VALUES(17,'Statistics'),
      (18,'Numerical Analysis'),
      (19,'Programming'),
      (20,'Website development'),
      (21,'Logistics'),
      (22,'AI'),
      (23,'Root-Finding Algorithms'),
      (24,'Newtonian Physics research');

INSERT INTO applies(cand_usrname,job_id,appl_state)
VALUES('iMaxinq',19,'Active'),
      ('Agaphmemnonas21',24,'Cancelled'),
      ('iMaxinq',20,'Active'),
      ('LokoLuke',21,'Cancelled'),
      ('ChrisP',23,'Active'),
      ('Marika365',17,'Active');

INSERT INTO degree(titlos,idryma,bathmida)
VALUES('Computer Engineering and informatics','University of Athens','PhD'),
      ('Mathematics','University of Thessaloniki','BSc'),
      ('Economics','University of Patras','MSc'),
      ('Business Administration','University of Athens','MSc'),
      ('Civil Engineering','University of Thessaloniki','MSc'),
      ('Physics','University of Thessaloniki','PhD');

INSERT INTO has_degree(degr_title,degr_idryma,cand_usrname,etos,grade)
VALUES('Computer Engineering and informatics','University of Patras','iMaxinq',2021,8.3),
      ('Mathematics','University of Thessaloniki','Nikolis54',2016,7.4),
      ('Economics','University of Athens','LokoLuke',2013,10),
      ('Business Administration','University of Athens','Marika365',2017,7.6),
      ('Civil Engineering','University of Thessaloniki','Agaphmemnonas21',2014,7.4),
      ('Physics','University of Thessaloniki','Nikolis54',2017,8.9);
```